Contents

Prerequisites

You will want to have GameMaker Studio 2 downloaded to follow along with this document. Additionally, you may want to check out the document *Navigating GameMaker - How to Find What you Are Looking For* to best follow along with this document.

Introduction

Video games are composed of many parts. On one part is what the player sees. Then, you have the internal logic for what happens. Finally, everything the player sees is contained in spaces that the player plays in. GameMaker Studio and other game engines break down these components separately. For GameMaker Studio, these components are sprites, objects, and rooms.

This document will provide a complete breakdown of sprites, objects, and rooms. There will be a walkthrough at the end of the document to put all the pieces together, during which you will create a screen saver.

Sprites

Sprites are two-dimensional visuals in video games. In GameMaker Studio, sprites are what you see. You can think of sprites as static pictures. They do not contain any internal logic, like movement or input.

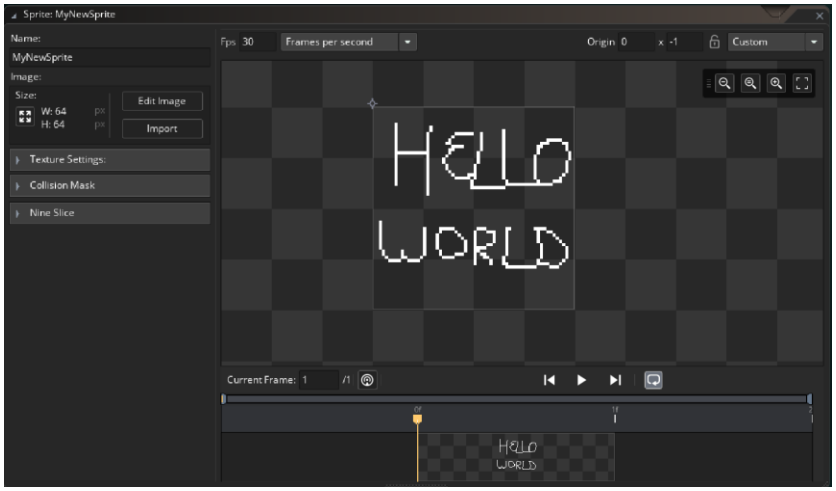

Figure 1 - Sprite Workspace View

You can create sprites directly in GameMaker Studio:

1. Open your GameMaker project and find your assets browser (typically on the right of the screen).

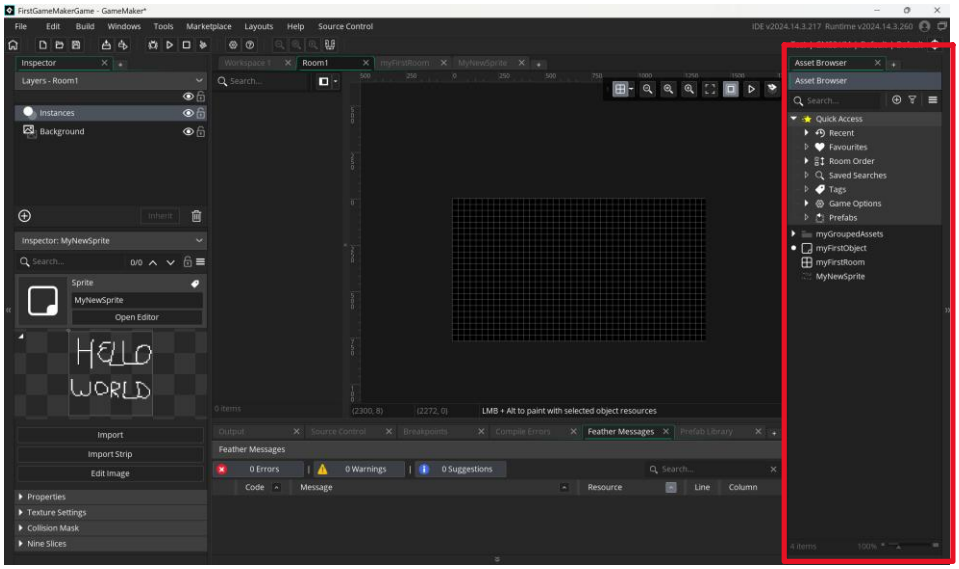

Figure 2 - Finding Assets Browser in GameMaker

2. Right-click on the empty space in the assets browser. Click Create->Sprite. Give your sprite a name, like “spr_myFirstSprite”.

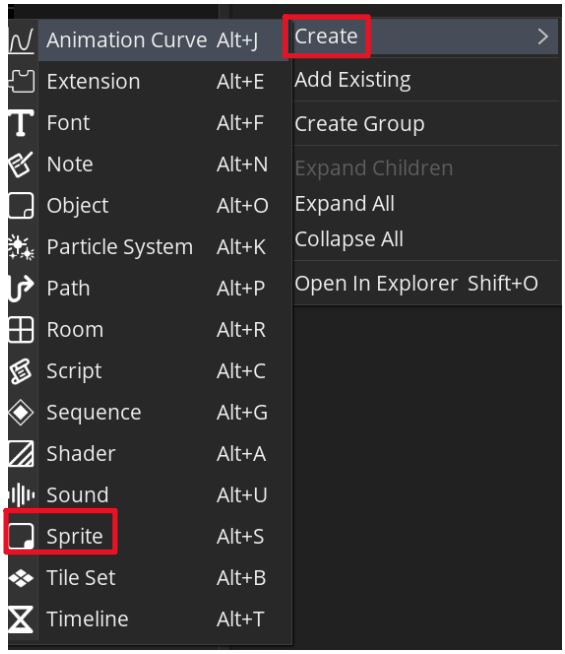

Figure 3 - Finding Create Sprite Location in Assets Browser

a. Note: It can be confusing to find different assets in your assets browser. However, using a naming convention at the beginning of the asset's name can help. For sprites, a common naming convention is to use "spr_" at the beginning to help you see at a glance that the asset is a sprite ("spr_" = sprite asset).

3. There are two ways to view your sprite: the inspector and workshop views. We will use the workspace view. Double-click on your sprite to open it in workspace view.

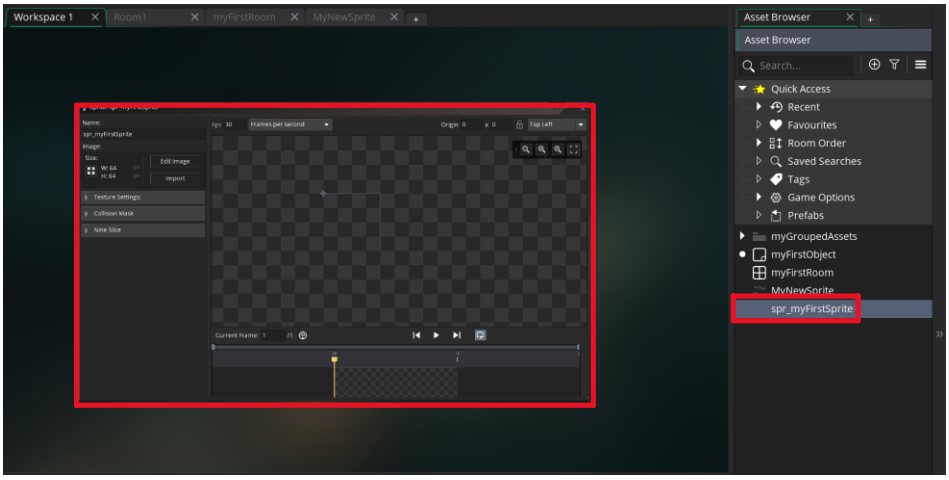

Figure 4 - Opening Sprite in Workspace View

There are many things you can do to sprites in your game. You can:

- Edit the size of the sprite

- Edit the image/import an image
- Apply texture settings
- Apply collision mask
- Use nine slicing
- Control its animation frame rate
- Set sprite origin

Editing Sprite Size

Editing the sprite size lets you change the sprite's length and width in pixel units. There are two modes: scale image and resize canvas.

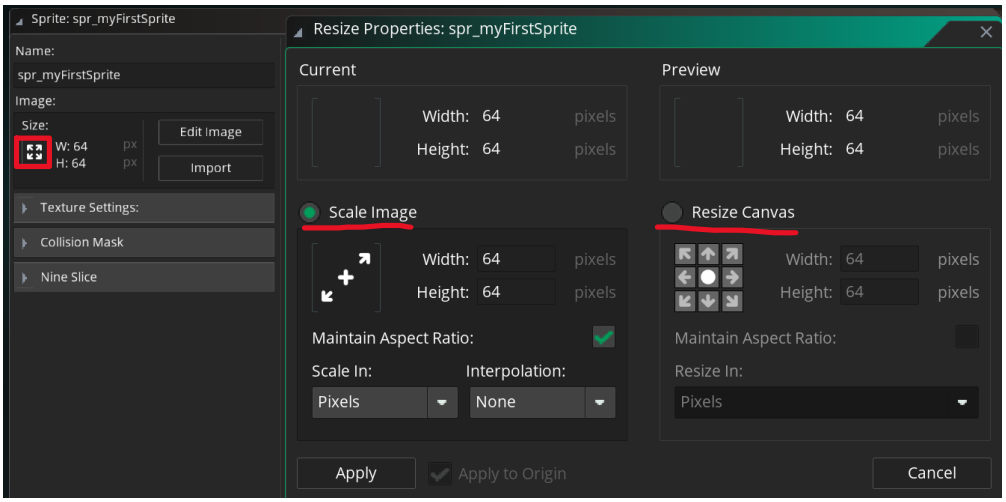

Figure 5 - Editing Image Size Options

Scaling the image means you take the sprite and stretch it to fit the new image size. You can scale by pixel count or by percentage. Either way, scaling will have the same effect, just done through a different system.

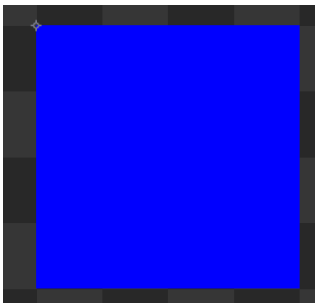

Figure 6 - 64x64 Cube Dimension

Figure 7 - Scaling Cube Length by 100%

Resizing the canvas gives you more space to draw your sprite. The default canvas size is 64 pixels by 64 pixels. You can change the canvas size to whatever you need for your sprite.

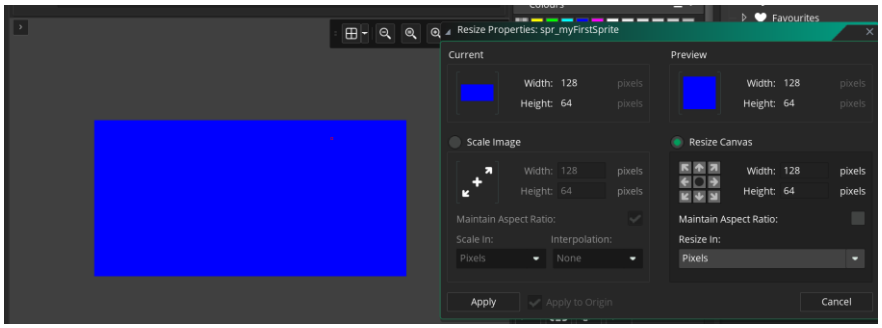

Figure 8 - Resizing Canvas Settings

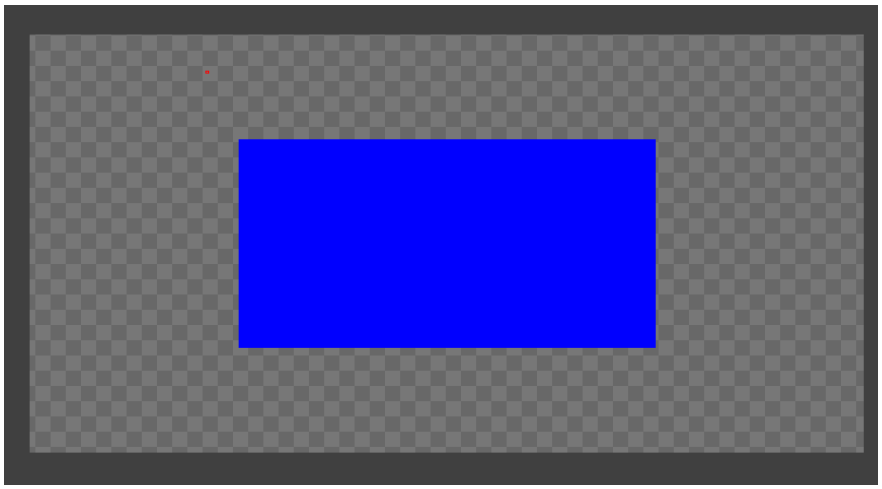

Figure 9 - Effect of Doubling Canvas Length & Width

Editing/Importing Sprites

You can edit sprites directly in GameMaker Studio. To do so, click Edit Image. This will open a workspace for you to edit the sprite.

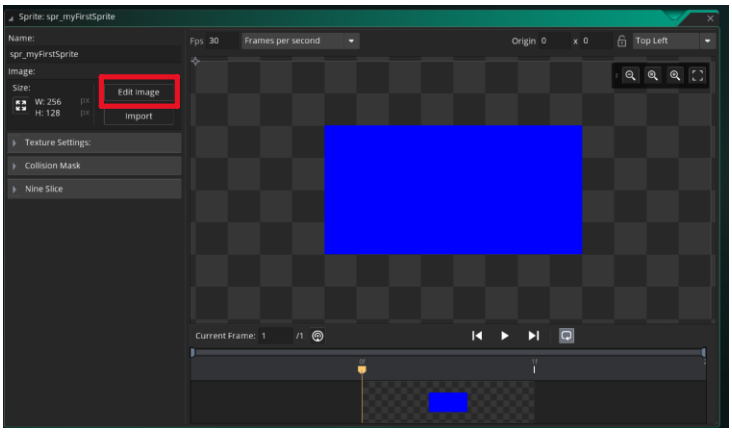

Figure 10 - Location of Edit Image in Sprite Workspace

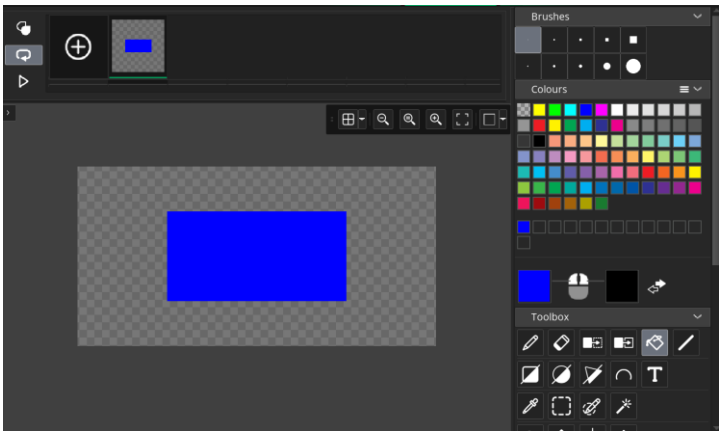

Figure 11 - Viewing The Edit Image Workspace

Alternatively, if you found or made a sprite outside of GameMaker Studio, you can import it. GameMaker Studio supports the following file types:

- PNG
- JPEG
- GIF
- BMP

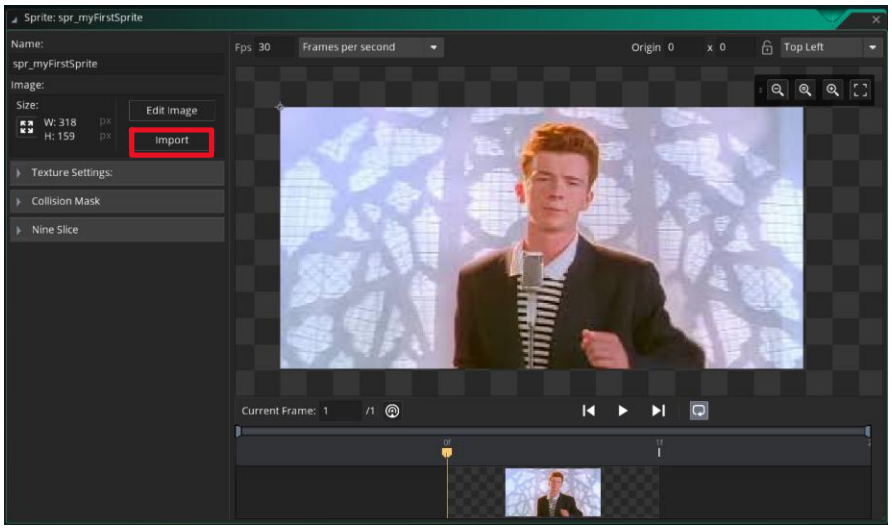

Figure 12 - Importing an Image of an Iconic Gentleman Into GameMaker

Texture Settings

Sprite texture settings let you control how GameMaker Studio stores and renders your sprites. Texture settings let you control how sprites are sent to and retrieved from your machine's GPU, and applying texture settings can help improve your game's performance, memory usage, and visual quality.

As a beginner, do not concern yourself too much with texture settings. Your game will run fine without messing with these settings. However, if you are making a large GameMaker project and want to improve visual performance, then texture settings are the place to start.

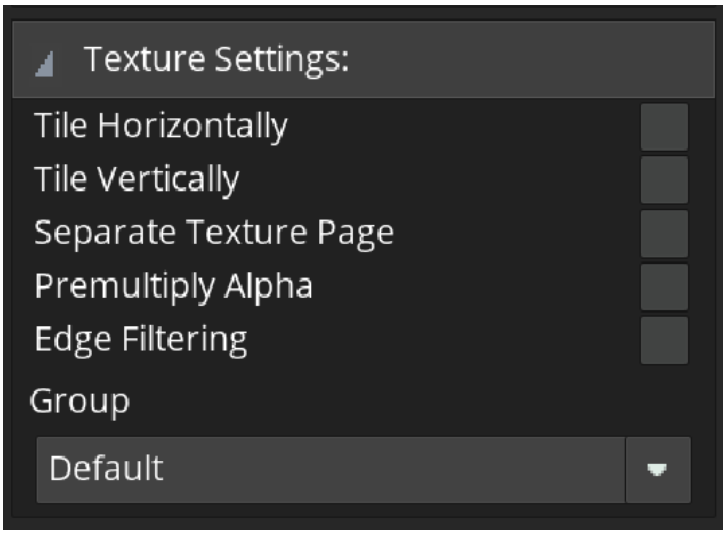

Figure 13 - Texture Settings Content

Collision Masks

The sprite collision mask allows you to control how it and other sprites will collide with each other. When opening the collision mask, you will see the collision area as a grey area on your sprite.

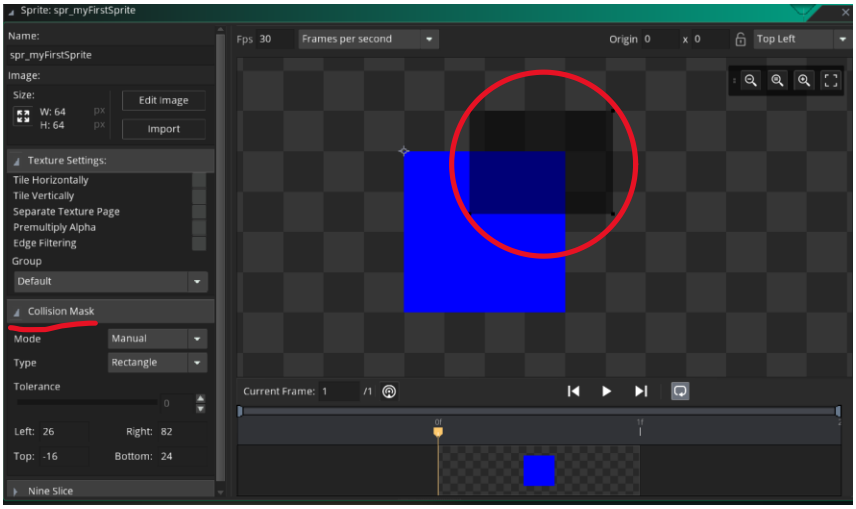

Figure 14 - Viewing Sprite's Collision Mask in Sprite Workspace

There are some options you can play with to set up your collision mask. You can let GameMaker create a collision mask for your sprite automatically by selecting automatic in the Mode dropdown.

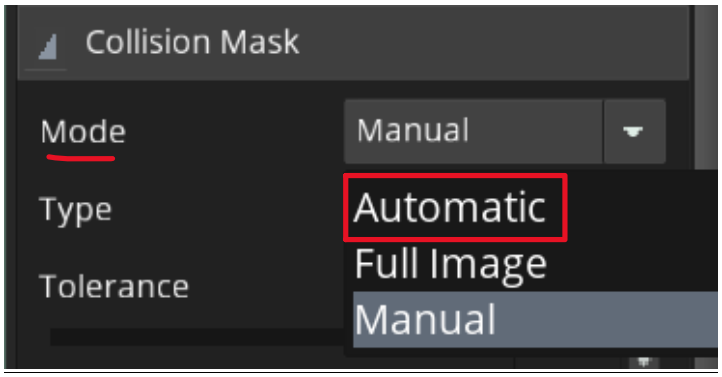

Figure 15 - Select Collision Mask Mode

Then, you can select the type of collision you want. Common collision types are rectangles, ellipses (circles), and diamonds.

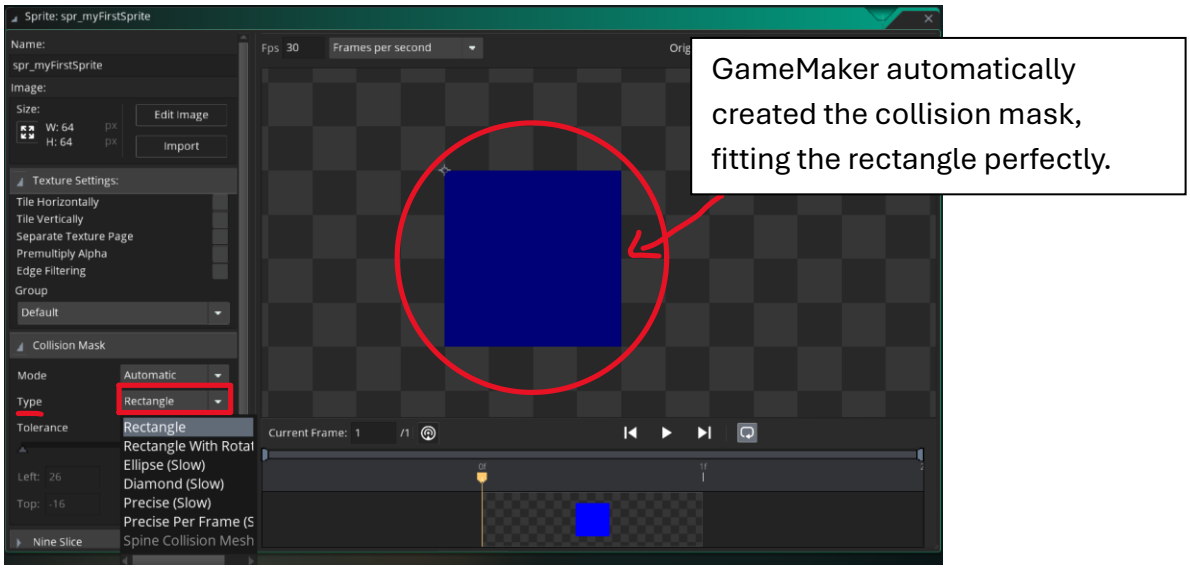

Figure 16 - Results for Setting Collision Mode Automatic, Type to Rectangle

If you want your sprite to interact with other sprites in your room, then you want to give it a collision mask.

Nine Slices

The nine slices to a sprite refer to the process of breaking up your sprite into nine pieces: four corners, four edges, and one middle.

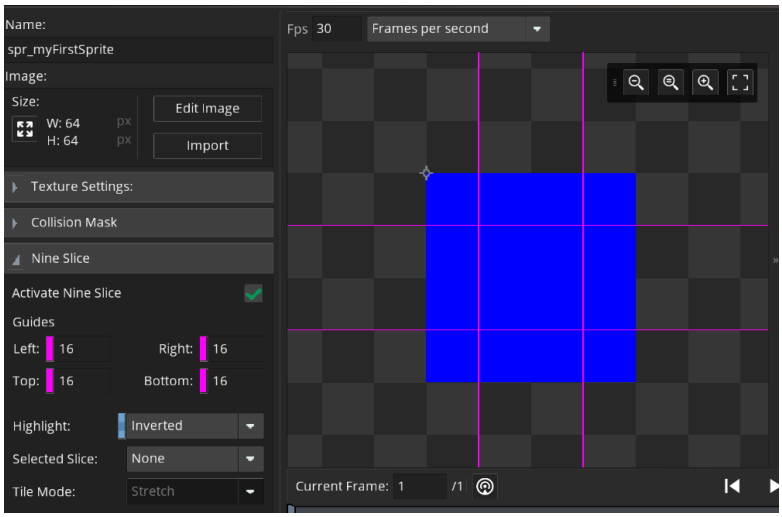

Figure 17 - Nine Slice View in Sprite Workspace

Nine slices can be used for UI sprites. Commonly, UI elements will change in size. Without configuring nine slices for your UI sprite, the corners and edges may stretch awkwardly and look distorted. When nine slices are properly set up, your sprite will scale and maintain a proper visual at any scale.

Frame Rate

A sprite's frame rate refers to how fast the sprite animation will play. If you have an animation set up (more on that later), the frame rate lets you control how quickly you shift from one sprite to the next one.

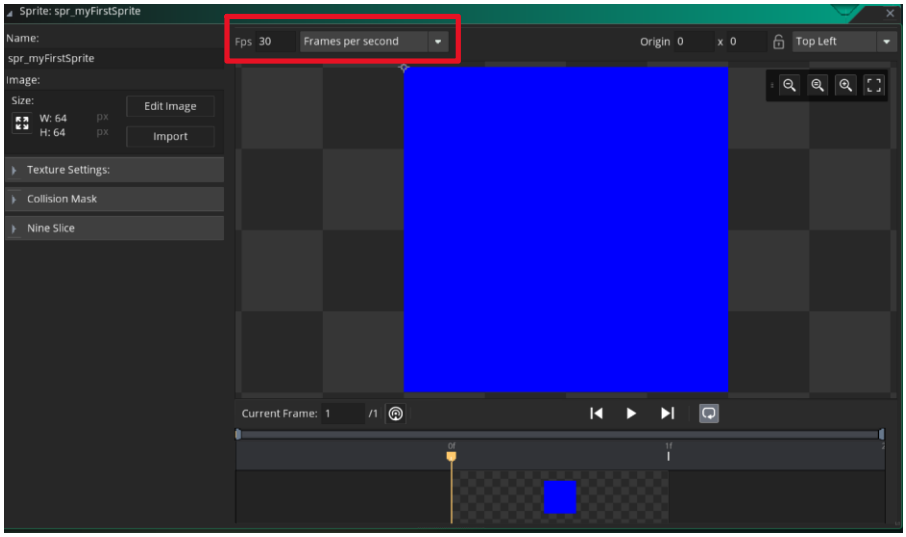

Figure 18 - Frame Rate Location in Sprite Workspace

For example, when your sprite's frame rate is set to 30 frames per second (fps), which is the default frame rate, then you will flip through your animation 30 times in one second, or one sprite flip every 0.033 seconds. You will get smoother motion by using higher frame rates (at the cost of performance), and you will get choppier motion by using lower frame rates.

Setting Sprite Origin

Setting the sprite's origin controls how basic transformations (movement, rotation, and scaling) apply to the sprite. Sprite origins use vector-based math a lot. If you wish to learn more about vectors, you will want to take MATH 149: Elementary Linear Algebra on campus.

You can set the origin for your sprite manually or automatically in GameMaker. The origin is depicted by a tiny crosshair.

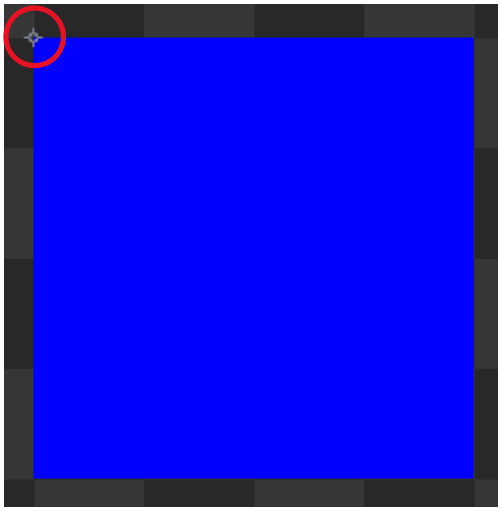

Figure 19 - Finding Sprite Origin in Sprite Workspace

You can change the origin manually by dragging the crosshair anywhere in the sprite's workspace. Alternatively, you can have GameMaker automatically position the origin with the dropdown at the top-right of the sprite workshop.

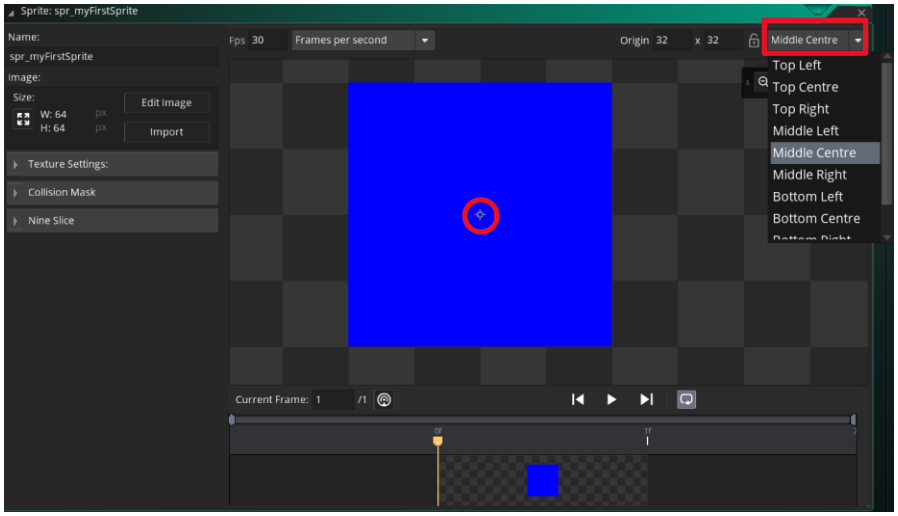

Figure 20 - Changing Sprite Origin

SpriteSheets

Spritesheets are a collection of sprites that define animations for a sprite. Each animation contains sprites that are played together to create the illusion of motion.

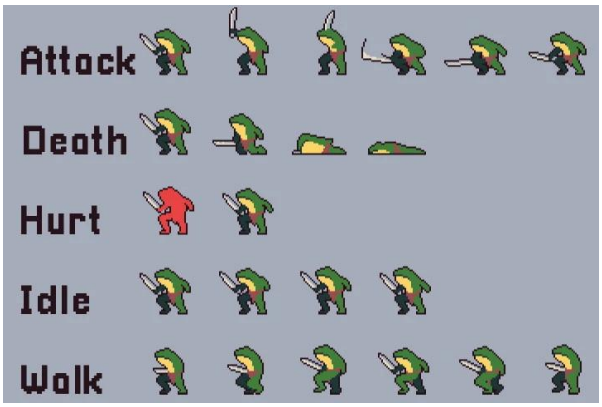

Figure 21 - Sample SpriteSheet

Figure 22 - Sample Sprite Animation

Importing A SpriteSheet

You can set up an animation in GameMaker using the sprite asset. If you have a spritesheet saved as an image, you may import it into GameMaker like you would any other sprite:

1. Click the import button, and open your spritesheet.

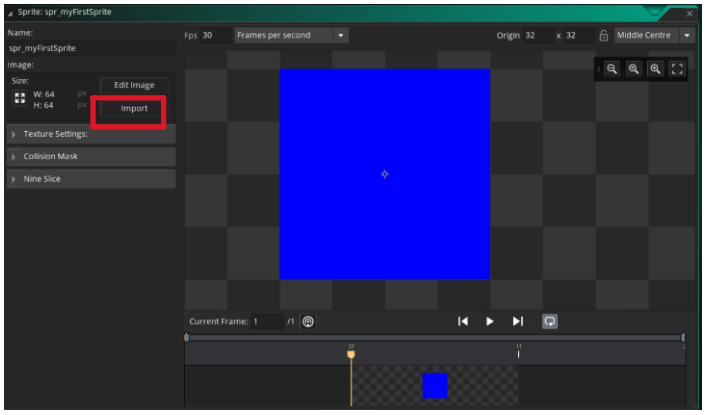

Figure 23 - Importing SpriteSheet Image To Sprite

2. Click on Edit Image.

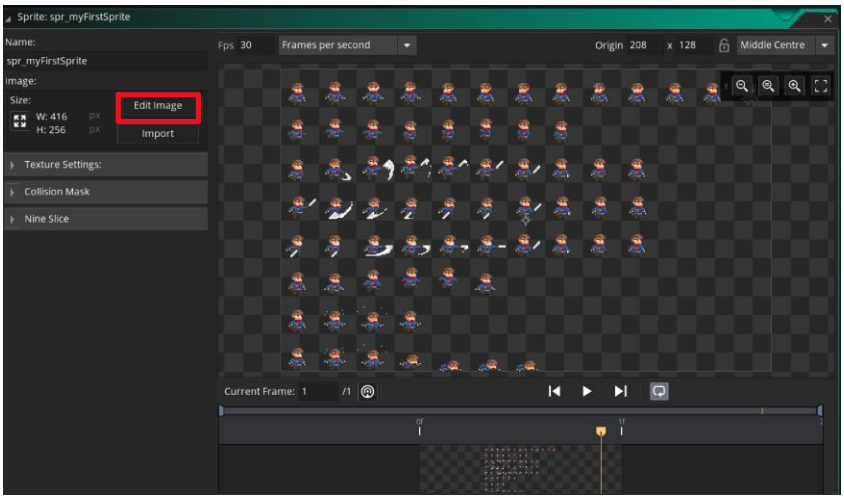

Figure 24 - Opening Edit Image on SpriteSheet

3. Navigate to the top toolbar. Click on Image -> Convert to Frames. This will open a menu where you can select your animation frames.

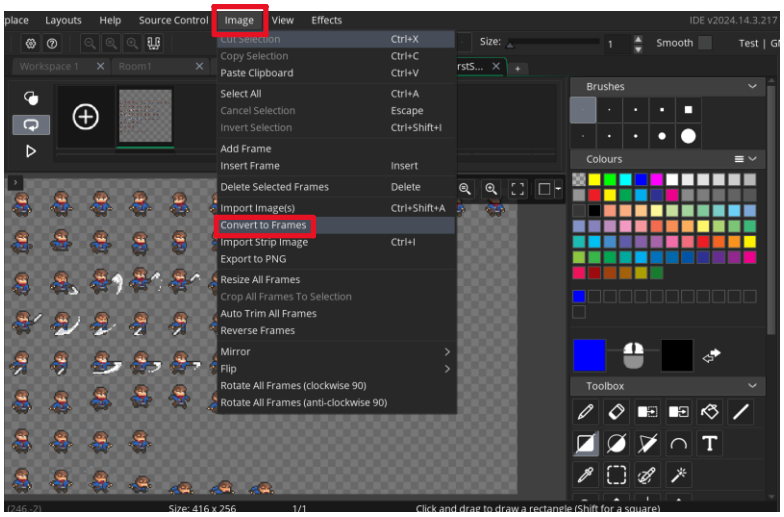

Figure 25 - Finding Convert to Frames Option for SpriteSheet

4. Set up the frame boxes to fit around the sprites that make up your animation. For example, in the illustration below, the first row was selected to get an idle animation.

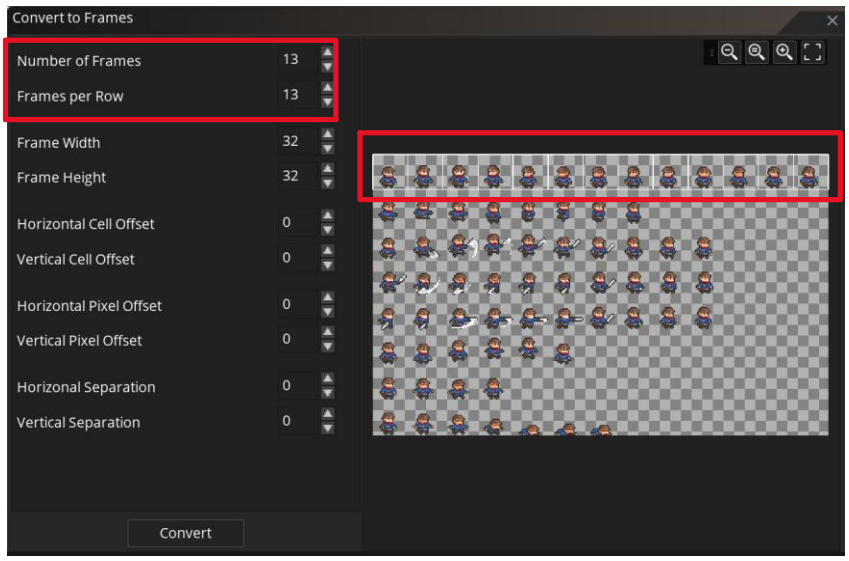

Figure 26 - Selecting Sprites in SpriteSheet

5. Convert the frames. This will place your sprites in separate frames.

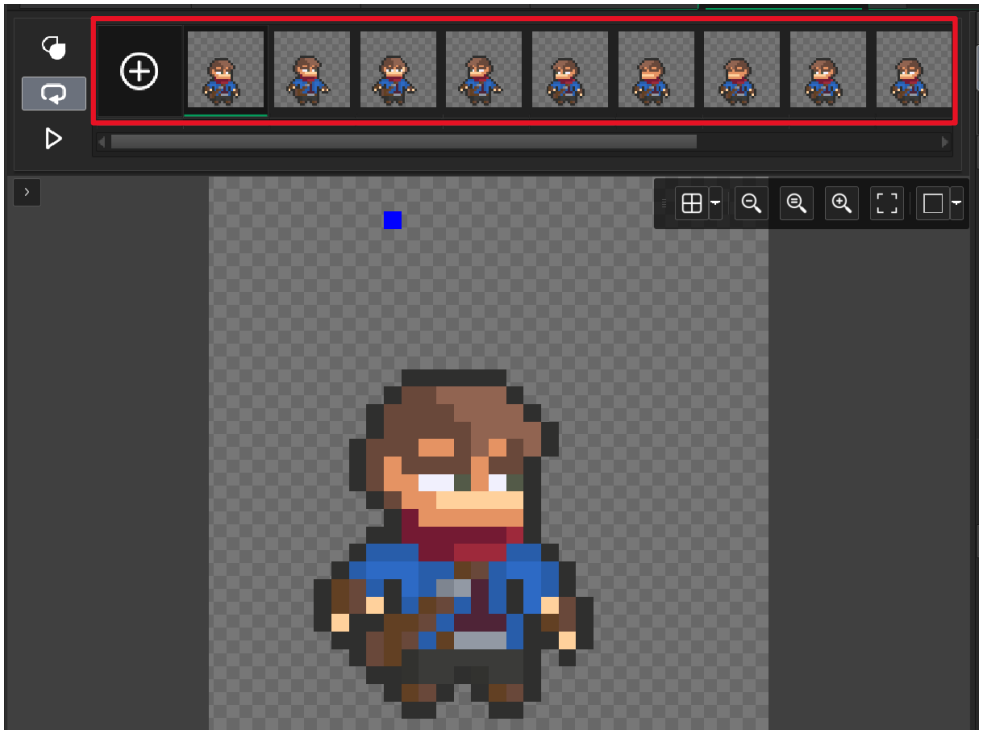

Figure 27 - Result of Converting SpriteSheet to Frames

6. To view your animation in the sprite editor, click the play button at the top of the sprite editor. Try running the animation with different frame rates.

Creating an Animation in GameMaker

You can also create animations in GameMaker itself.

1. Open your sprite editor.
2. At the top of the editor, you will see a + button. Clicking it will create a new frame for your animation.

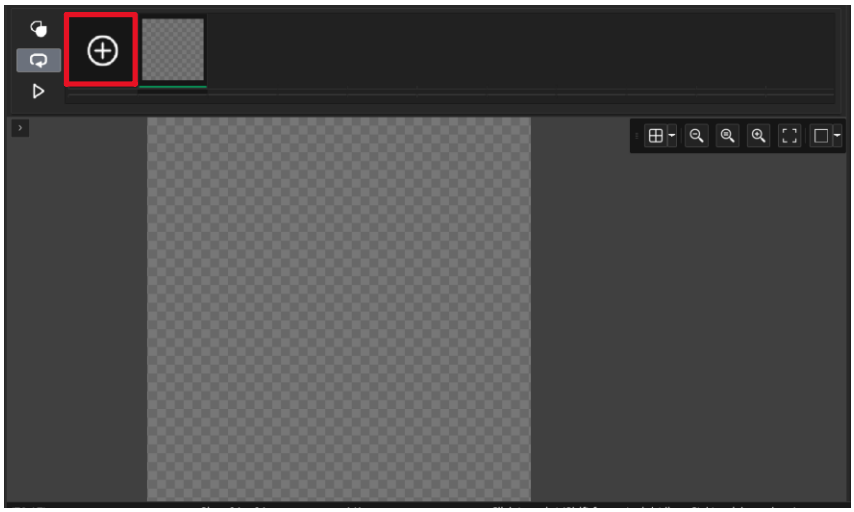

Figure 28 - Finding Location to Create New Frames in Sprite Workspace

3. Try creating an animation with your sprites. Once done, click the play button to view the animation. Adjust your frame rate if necessary.

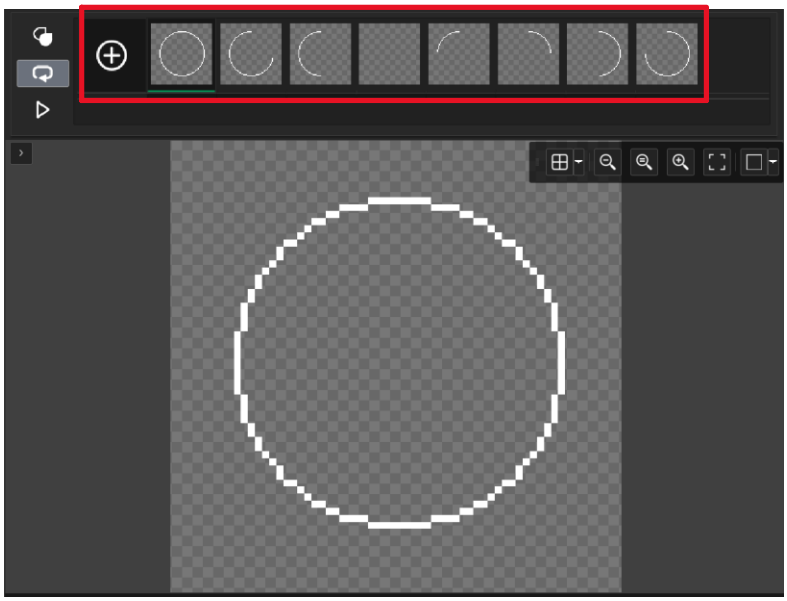

Figure 29 - Sample Custom Sprite Frames

Objects

Objects are the core interactive elements in your game. Objects are the building blocks of logic. You can set up object behavior to do practically anything you want in your game.

Objects have many properties, including:

- A sprite
- Event behavior
- Physics
- Collision mask
- Variable definitions
- Room persistence

These properties define how your object looks and behaves in your game, and GameMaker offers versatile support for each of these properties.

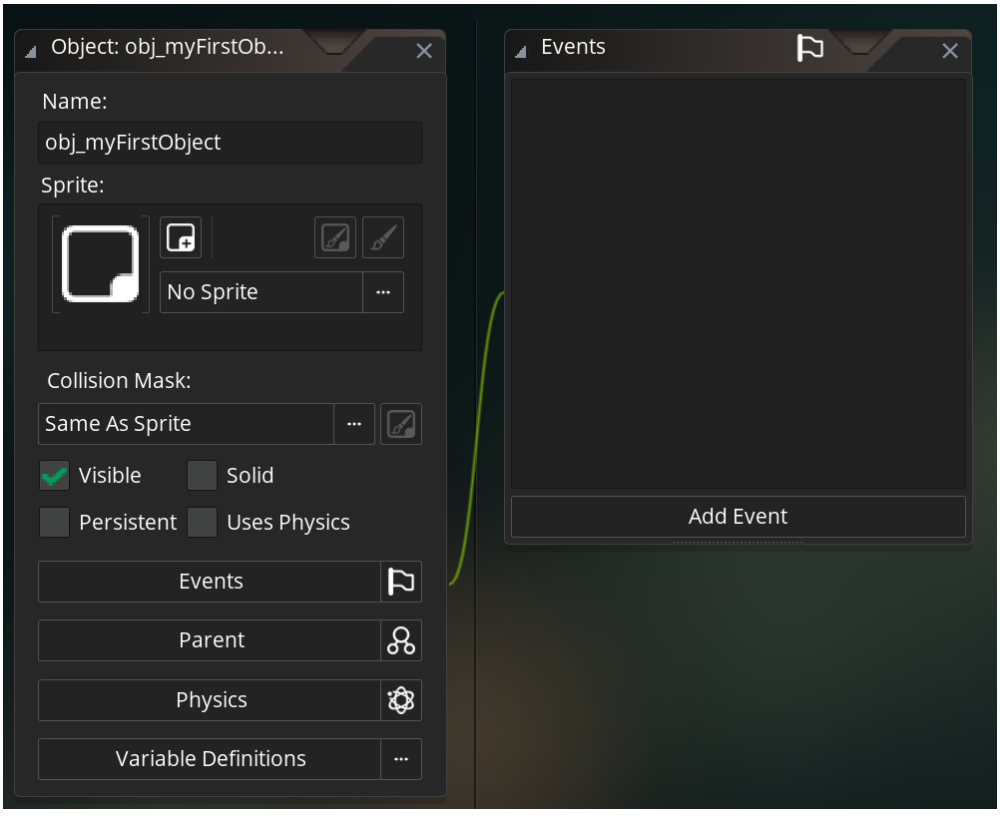

Figure 30 - Object View in Workspace

A Quick Distinction Between Object Assets & Object Instances

When discussing objects, it is critical to understand the distinction between an object asset and an object instance. In GameMaker Studio, object assets are the building blocks for object instances. Object instances are the *real* instances of an object asset that exist in your game.

Consider the following: you are constructing many similar houses for a new town. You make a blueprint that lays out the house's dimensions, rooms, walls, etc., to help build the house. Afterward, you share the blueprint with the construction workers to make the house. In this analogy, the blueprint is your object asset, and the houses are the instances of said object.

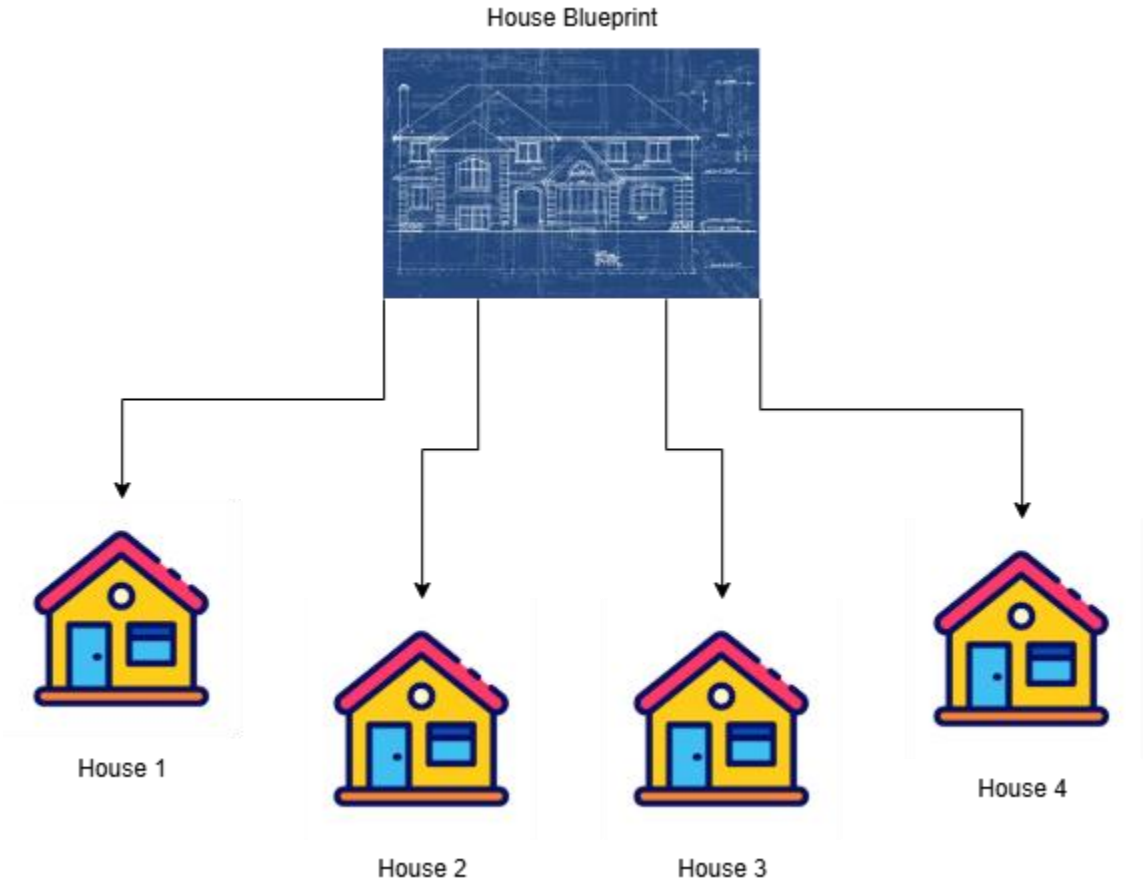

Figure 31 - Object Asset vs. Object Instances Diagram

In game development, the process of creating an object in your game is called **instantiation**. In general, every time you add a game object to one of your rooms, you are instantiating an object asset into existence.

Object Sprite Property

The object's sprite property is the visual representation of your object in your game. Conversely, it is where you can bring your sprites to life through logic. You can set a default sprite to your object in the object workspace by clicking on the sprite selection element in your object's work view.

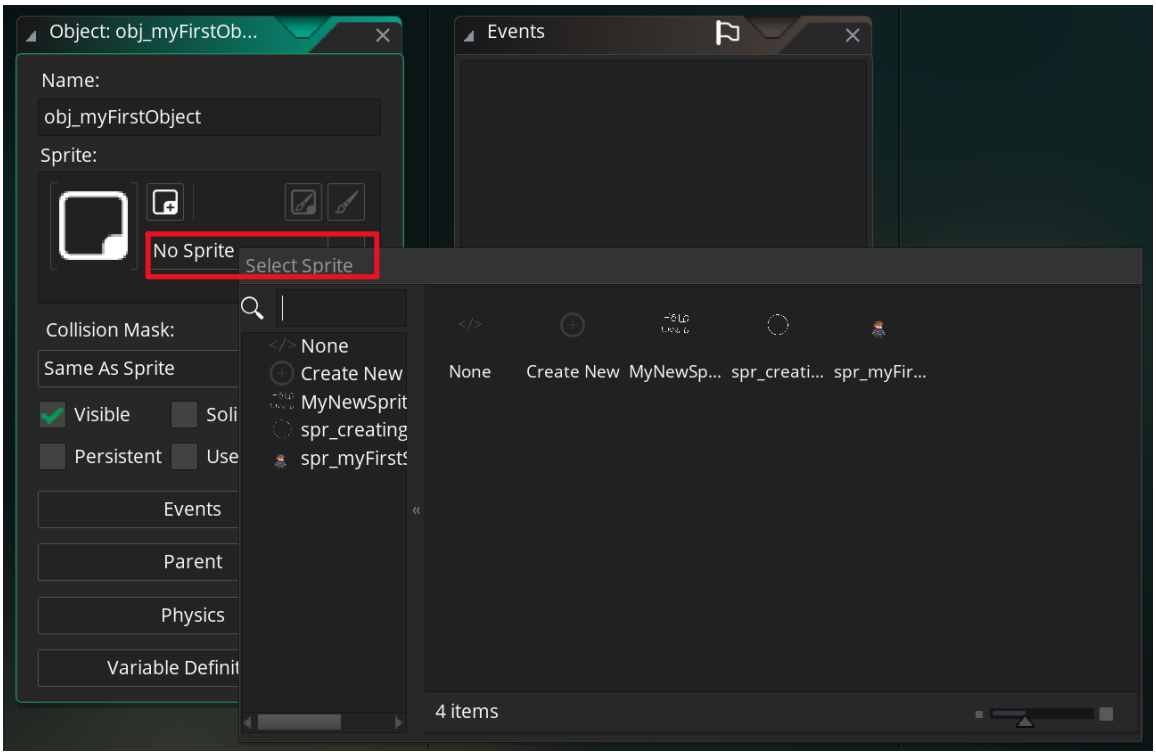

Figure 32 - Location to Add a Sprite to an Object

Then, when you instantiate the object, you can see it in your game.

The object's sprite property can also hold a sprite animation. Your animation will play by default. If you want to prevent your sprite from playing by default, you will have to control the animation speed in the event behavior.

Event Behavior

An object's event behavior is where its logic is configured. This is where you can code your object to do whatever you want. You can find your object's event behavior by clicking on the Events button, which will open up a side view for all of your object's events.

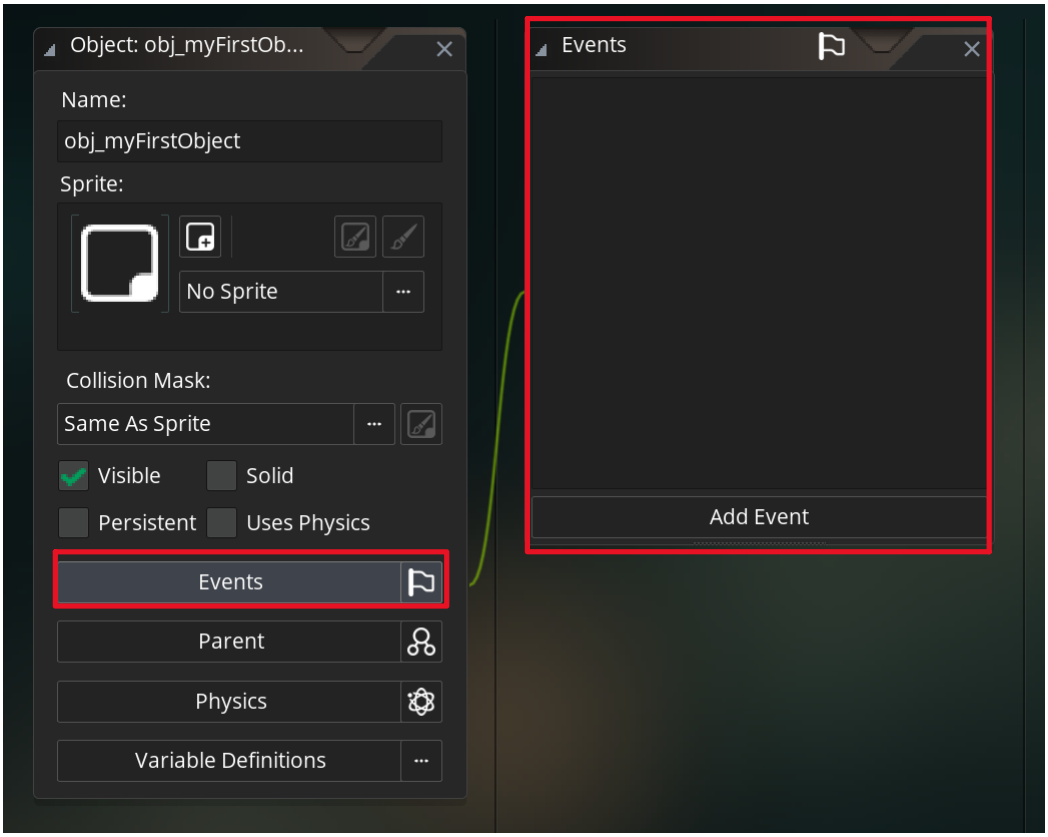

Figure 33 - Finding an Object's Event Behavior in The Object's Workspace

GameMaker Studio offers two ways to program events. You can either use text-based code or GameMaker's built-in drag-and-drop system. If this is your first time coding, I recommend using the drag-and-drop system. However, I also recommend learning text-based coding, as it provides full control over your object's behavior and can be easier to read for larger projects. You can find many courses on campus to learn text-based coding, including CITA 140, 210, 225, 255, 350, and 355.

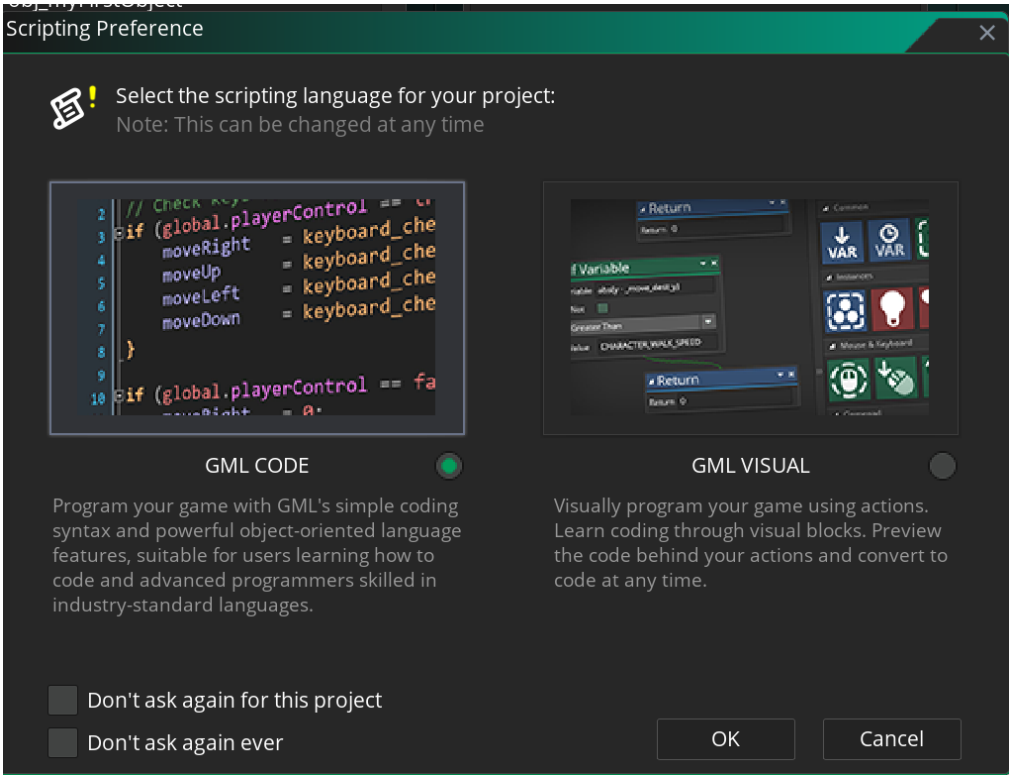

Figure 34 - GameMaker Scripting Language Popup Window

There is a lot to cover with events. For now, all you need to know is that an object's events are triggered based on things that happen in your game. When an event happens, the code you write for that event will process.

To learn more about GameMaker's event system, check out the document *An In-Depth Guide to GameMaker's Event System*.

Physics

GameMaker uses a built-in, 2D physics system. When physics are enabled for your object, it behaves as if it has a physical body in your world. This means it can dynamically interact with other physics bodies for collisions and friction.

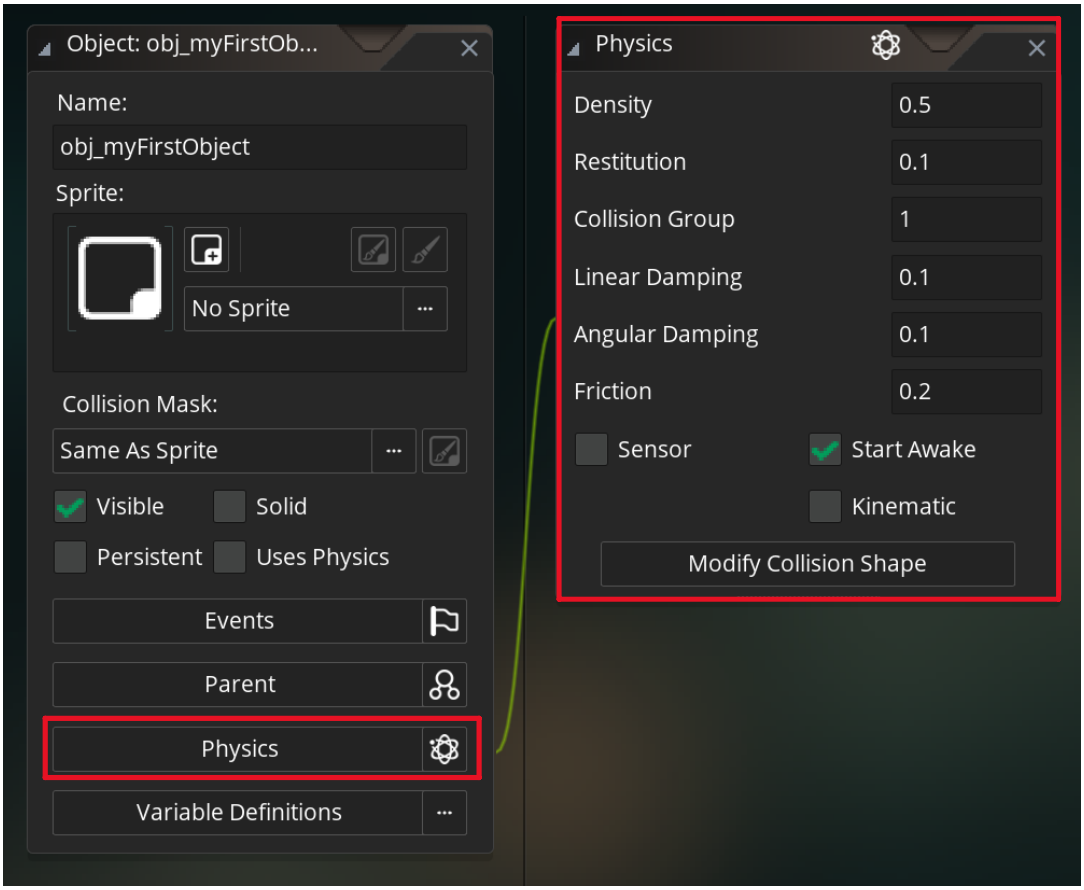

Figure 35 - Finding Physics Settings for Object in the Object's Workspace

Notice that the physics units are not labeled. In GameMaker, you decide the unit of measurement for physics. Just be sure to use the same unit of measurement across your entire game for accurate physics; you don't want to apply forces in newtons while your bodies are weighed using pounds. If you are using newtons as your unit of measurement, use newtons for your object's weight.

Density

The density physics property controls your body's mass. An object with a higher density value will be harder to push than one with less density. Conversely, a body with lower density will be easier to push. Think of density as your object's resistance to movement. If you want your object to resist movement more, increase its density.

Restitution

The restitution physics property controls how bouncy your object's body is, scaled from 0 to 1. A restitution value of 0 means your body will never bounce, whereas a restitution value

of 1 will make your body bounce with lossless forces; momentum is completely conserved when bouncing.

Collision Group

The collision group property is one setting where you can directly control what your object can and cannot collide with. When you put two objects in the same group number, you are telling GameMaker that they should either always interact or never interact. As a rule:

- Two objects with the same negative collision group number will *never* interact
- Two objects with the same positive collision group number will *always* interact
- An object with a collision group number of 0 will not use the collision group value to determine collision.

It's important to note that collision groups apply only when both objects have physics enabled and use the same collision group number. Otherwise, the collision between both objects is determined based on their collision category and mask.

Linear Damping

The linear damping physics property controls your body's air resistance when moving. A higher linear damping value will slow your object *more* than a lower one. If you don't want your body to float around while airborne, setting a linear damping value will gradually slow it down over time.

Angular Damping

The angular damping physics property controls how resistant your object's body is to rotational movement, also known as torque. Angular damping will gradually slow down the object's rotation.

Fr*iction*

The friction physics property controls how resistant your body is to movement when colliding with another object. If your object is sliding across another object, friction will occur, and your object will slow down based on the friction value. A higher friction value will slow your ball down *faster* than a lower friction value. A friction value of 0 means your body will slide with lossless forces; momentum is conserved while sliding against objects.

Sensor

The sensor physics property lets you control whether your object collides with other objects. When the sensor is set to true, your body will ghost through other bodies. However, your object will continue to receive collision information. Thus, you can apply the sensor

property when you want your object to receive collision information, but you don't want it to interact with other objects physically.

Kinematic Body

The kinematic physics property controls whether or not other objects can push your object's body. If you want your object to move without being pushed around, like a moving platform, then set kinematic to true.

Static Body

Unlike kinematic objects, static objects cannot move whatsoever. You cannot force it to move through code, and objects that collide with it will not push it. Static objects are typically walls and floors. You can make an object static by setting the body's density value to 0.

Dynamic Body

Dynamic physics bodies are what you would expect from a physics object. Dynamic objects can move freely and can be pushed around by other bodies. If your object is not set to kinematic or made static, then it is dynamic by default.

Collision Mask

The collision mask property for an object defines how it will collide with other objects in your game. You can set up the collision mask in the select sprite's properties (See Sprite section above).

Variable Definitions

Variable definitions are where you can store information about your object. Then, you can reference those variables in your object's event behavior later. For example, an enemy would have variables for move speed, health, and damage. In your code, you can call on these variables to get their value.

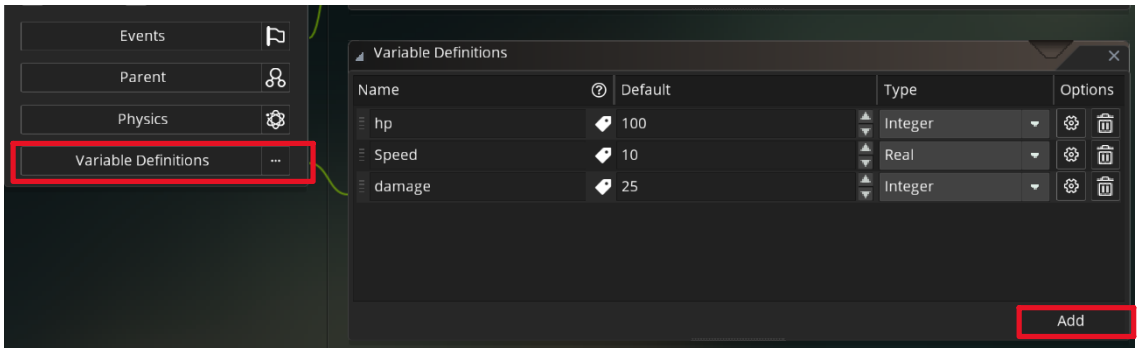

Figure 36 - Finding Variable Definitions in The Object's Workspace

Variable definitions let you write cleaner code for your game. Say you are using move speed in three separate areas in your code. Without variable definitions, you will have to find the location for each reference to your move speed and manually change its value to the new value. With variable definitions, you only need to change the speed value once, and all references to the move speed will update alongside it.

Room Persistence

Room persistence changes how your object behaves when moving between rooms in GameMaker. By default, room persistence is disabled. When disabled, your object will be destroyed before GameMaker moves your game to a new room. When enabled, your object will survive the room transition.

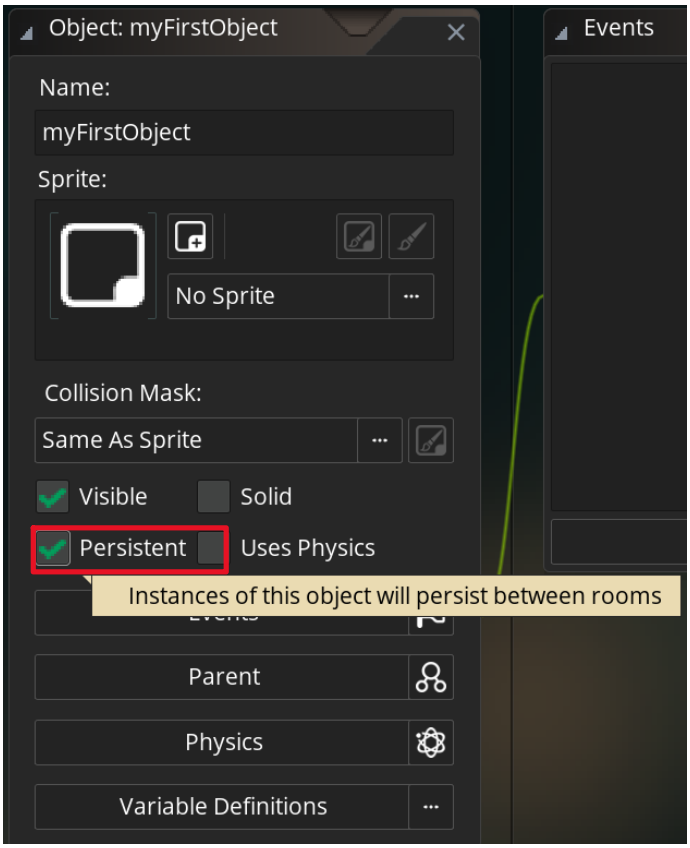

Figure 37 - Finding Room Persistent Setting in Object Workspace

In game development, objects that are room-persistent are often called singletons or managers. Typically, a room-persistent object has only one instance and is used for a single task. For example, you can have a singleton that manages your audio settings, like

music and sound effects volume. You only need one instance of an audio manager to control volume, and you want it to exist in every room in your game.

Rooms

In GameMaker, rooms are your game's world. Rooms are where the objects in your game can interact with each other, and the player can play the game. They hold information about the game's layout or level.

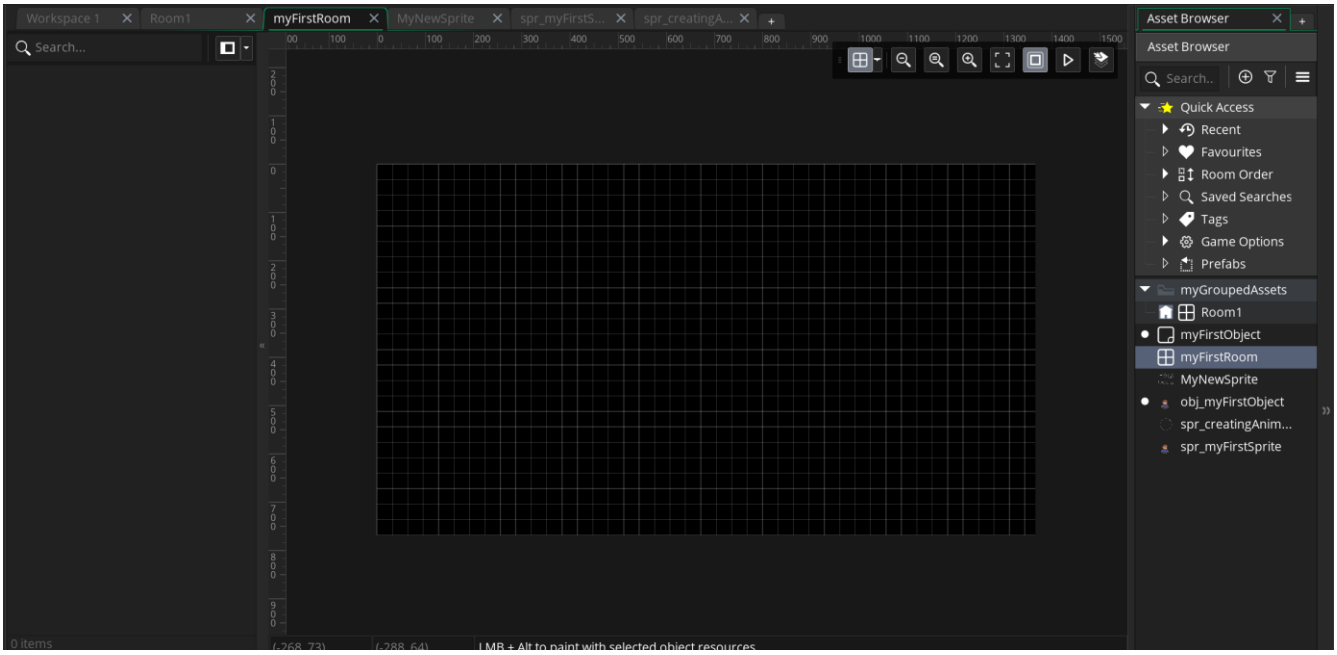

Figure 38 - Viewing Room Workspace

GameMaker's rooms are navigated through a 2-dimensional coordinate system (x, y). You can break room movement into left/right and up/down movements. Left/right movement corresponds to your x-value, and up/down movement corresponds to your y-value. In GameMaker, moving *right* will increase your x-value, moving *down* will increase your y-value, and vice versa. It may feel unintuitive for the y-value to increase when you move downward and for it to decrease moving upward; that's just how GameMaker works.

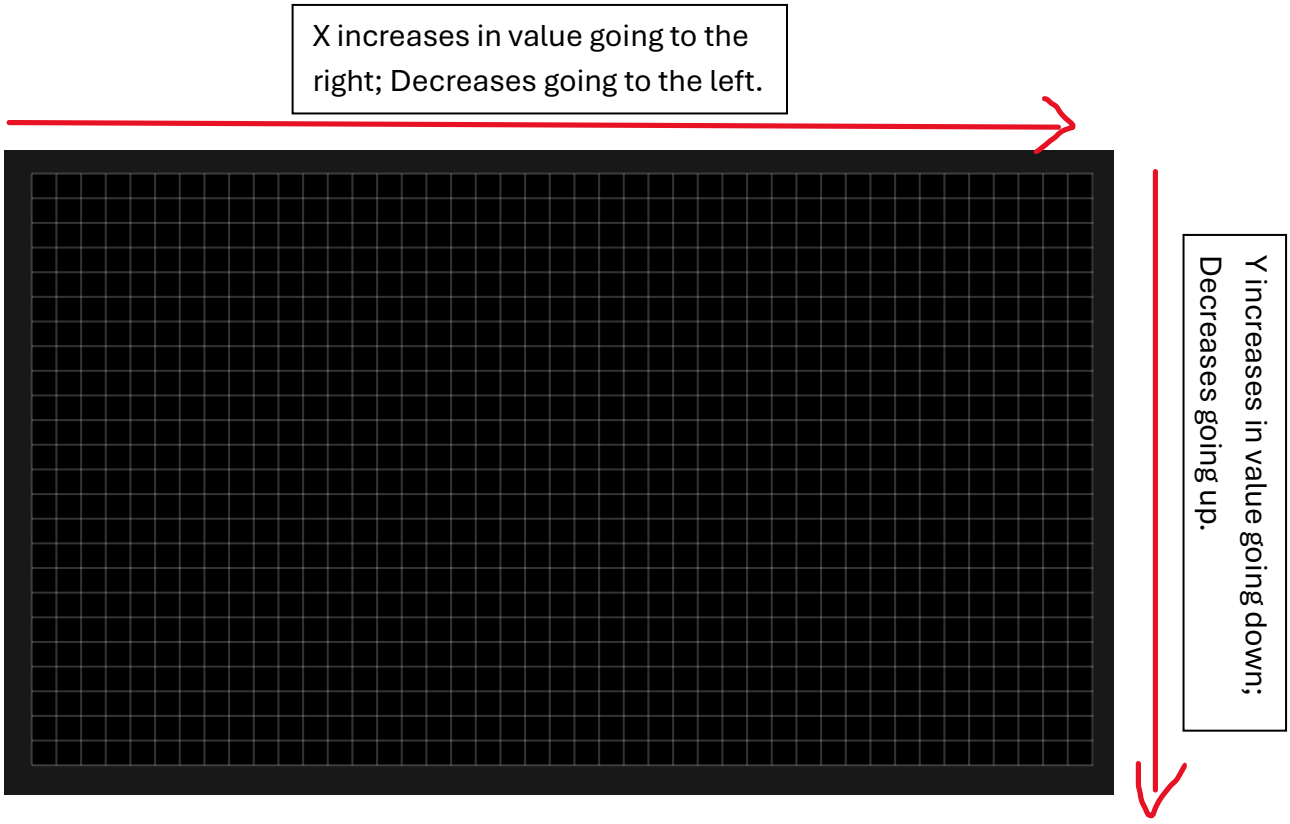

Figure 39 - Navigating Room Coordinate System

GameMaker has a room order that determines the order in which your rooms appear. Your game will always run the room at the top of your room order list. You can view the room order in the assets browser. More importantly, you can rearrange the room order by dragging the room assets to a new position in the list. The room that will be opened upon running the game is displayed with a house icon next to it.

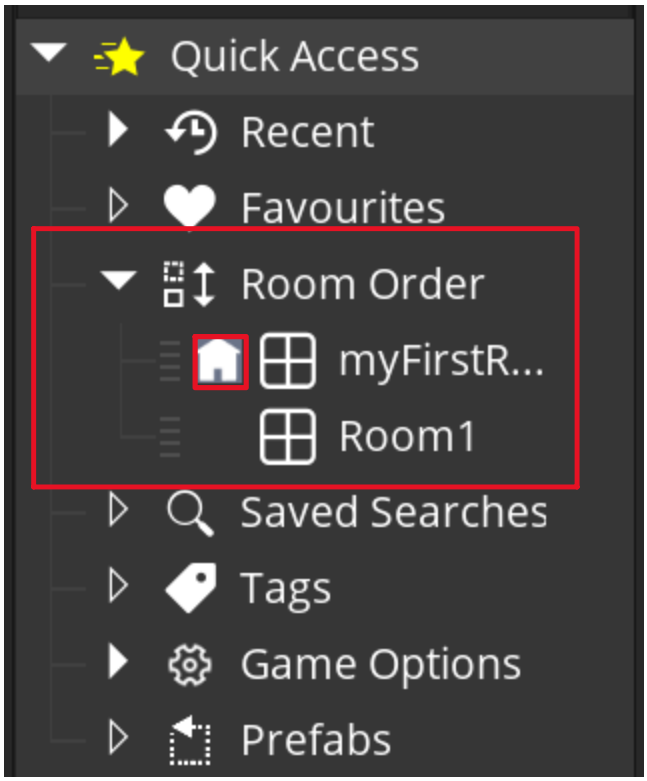

Figure 40 - Finding Room Order in Assets Browser

The Room's Inspector

The properties of each room asset can be viewed in the inspector window. Click on your room, then view the inspector's contents. The inspector should look similar to the following:

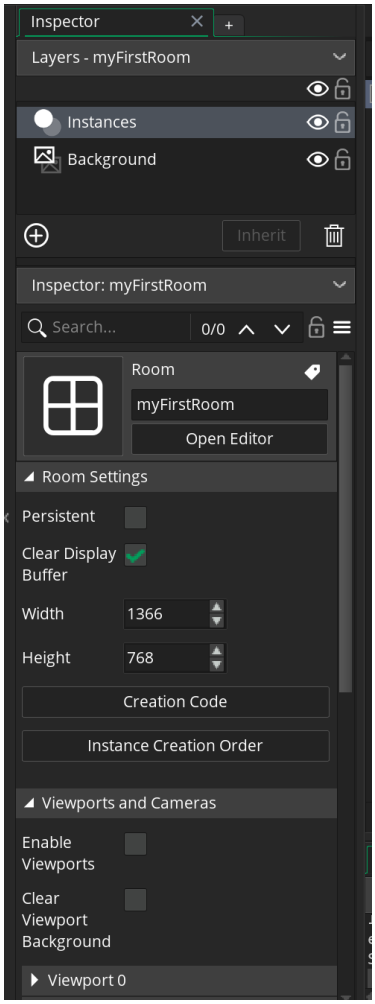

Figure 41 - Sample Inspector Properties for Room

Room Settings

The important room settings to know are the width and height. The width and height will control how long and tall the room is, respectively. You can give rooms different dimensions, and you can be creative with how you use room dimensions for your game.

Viewports & Cameras

In this section, you can set up your viewport settings. Viewports in GameMaker are displays of the room where your game takes place. You can think of viewports and cameras in GameMaker like security monitors. The monitor shows multiple displays, and in each, you can see the environment being recorded.

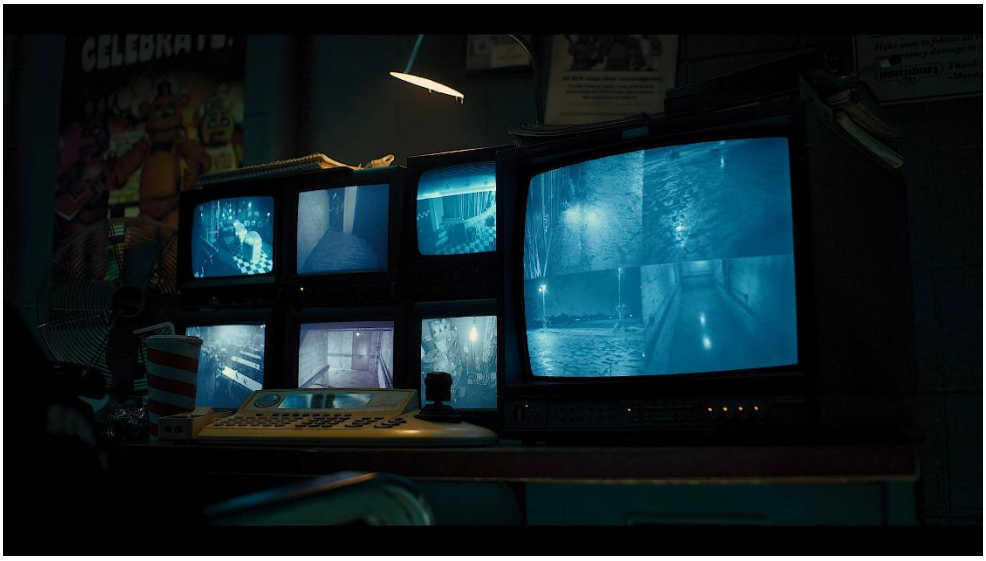

Figure 42 - Viewport vs. Camera Analogy

In this analogy, each display on the monitor is a viewport, and what the viewport shows comes from a camera. Thus, cameras are responsible for what is seen in your game, and viewports are responsible for determining how and where it will be shown to the player's screen.

Room Physics

The room's physics settings determine whether the room uses physics. Additionally, you can set up the gravity of your room, or a constant tug on your object's bodies in a given direction. Each room can have its own gravity, and you can make the gravity go in any 2D direction (up, down, left, and/or right).

Putting It All Together: Making A Screen Saver

There was a lot to talk about with sprites, objects, and rooms. Let's work through a simple example to put these concepts together. In this example, we will make a simple screen saver. We will make a simple logo that bounces off the walls of our room, changing color and direction each time it hits a wall.

Here's how it will work:

- We will have one object in our room, and we will give it a random diagonal direction.
- Every step/frame of the game, we will check if the surrounding object's collider hits the border of the wall
- If our object hits a wall, we will do the following:
    - Reverse the object's movement:
        - If we were going up and we hit the upper room border, then we will move down
        - If we were going down and we hit the lower border, then we will move up
        - If we were going left and we hit the left border, then we will move right
        - If we were going right and we hit the right border, then we will move left.
    - Update our screen saver image to a new color

Before getting started, there are some important variables built into objects that we want to be aware of:

- bbox_top: refers to the top-most point of our object's collision mask
- bbox_bottom: refers to the bottom-most point of our object's collision mask
- bbox_left: refers to the left-most point of our object's collision mask
- bbox_right: refers to the right-most point of our object's collision mask
- room_width: refers to the pixel width of our room. Also refers to the right-most side of our room
- room_height: refers to the pixel height of our room. Also refers to the bottom side of our room.
- vspeed: refers to the vertical speed of our object.
- hspeed: refers to the horizontal speed of our object.
- image_index: refers to the current frame of our sprite's animation. Indexing starts at 0, so the first frame has an index of 0, and so on.

- image_number: refers to the total number of frames for our sprite animation

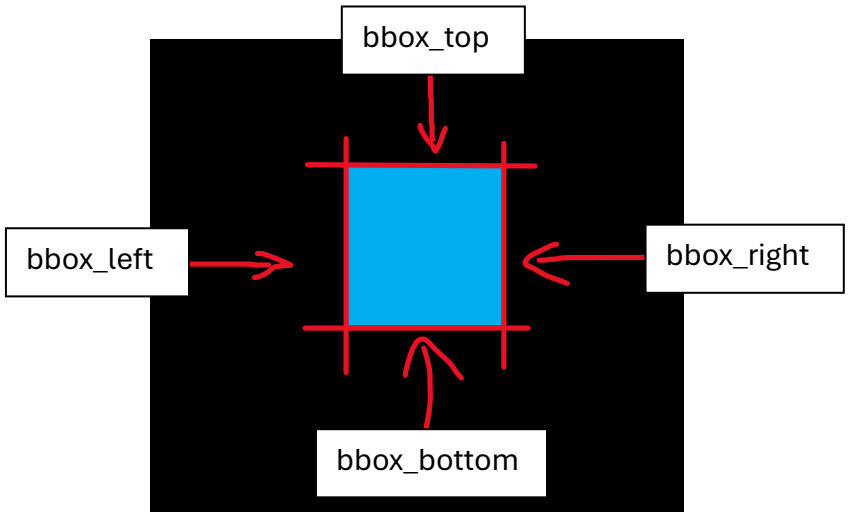

Figure 43 - Location of Object's bbox Properties

Figure 44 - Room Width & Height Locations in The Room

Figure 45 - Image Number & Image Index Sample Values

1. If you have not already done so, create a GameMaker project. Name it “ScreenSaver Project”, or something else of your choice, and choose a location to save your project on your computer.

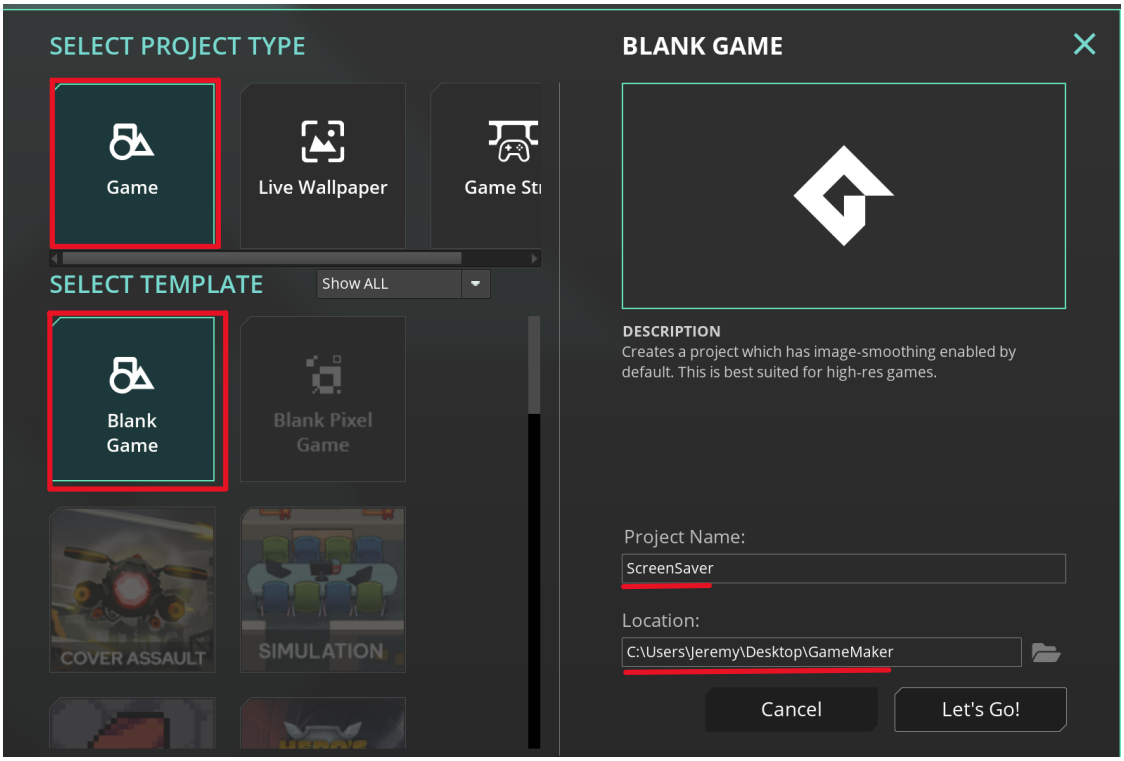

Figure 46 - Creating a New GameMaker Game

2. Create a new sprite in your assets browser. Name it “spr_Logo”.

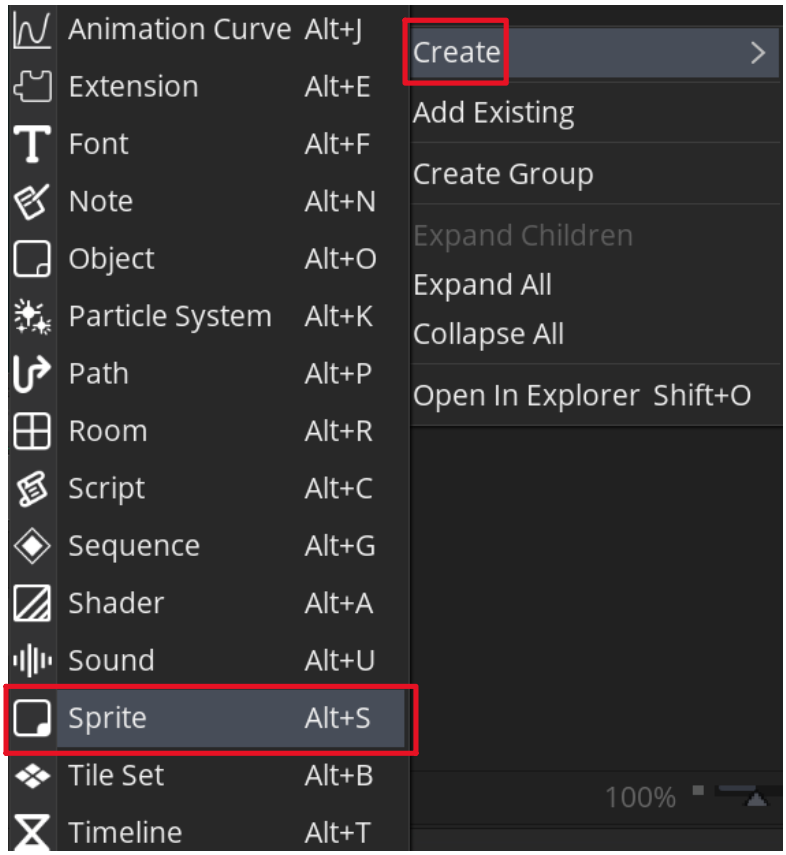

Figure 47 - Creating Logo Sprite

3. Double-click your new sprite asset to open it in your workspace. Then, click Edit Image. We will set up some basic frames for our sprite, each one for when it bounces off a different wall.

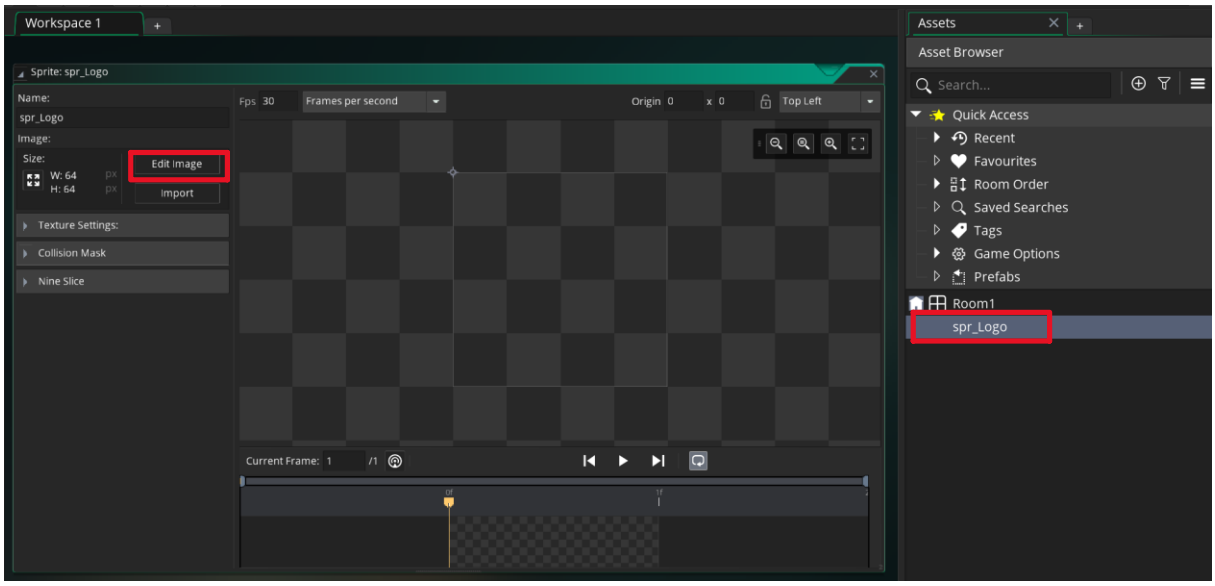

Figure 48 - Opening Sprite Logo in Workspace View

4. Create a couple of frames for your sprite. Try to keep each frame relatively similar in shape, but different in color. We will iterate through our sprite's frames each time it bounces off the room's border.

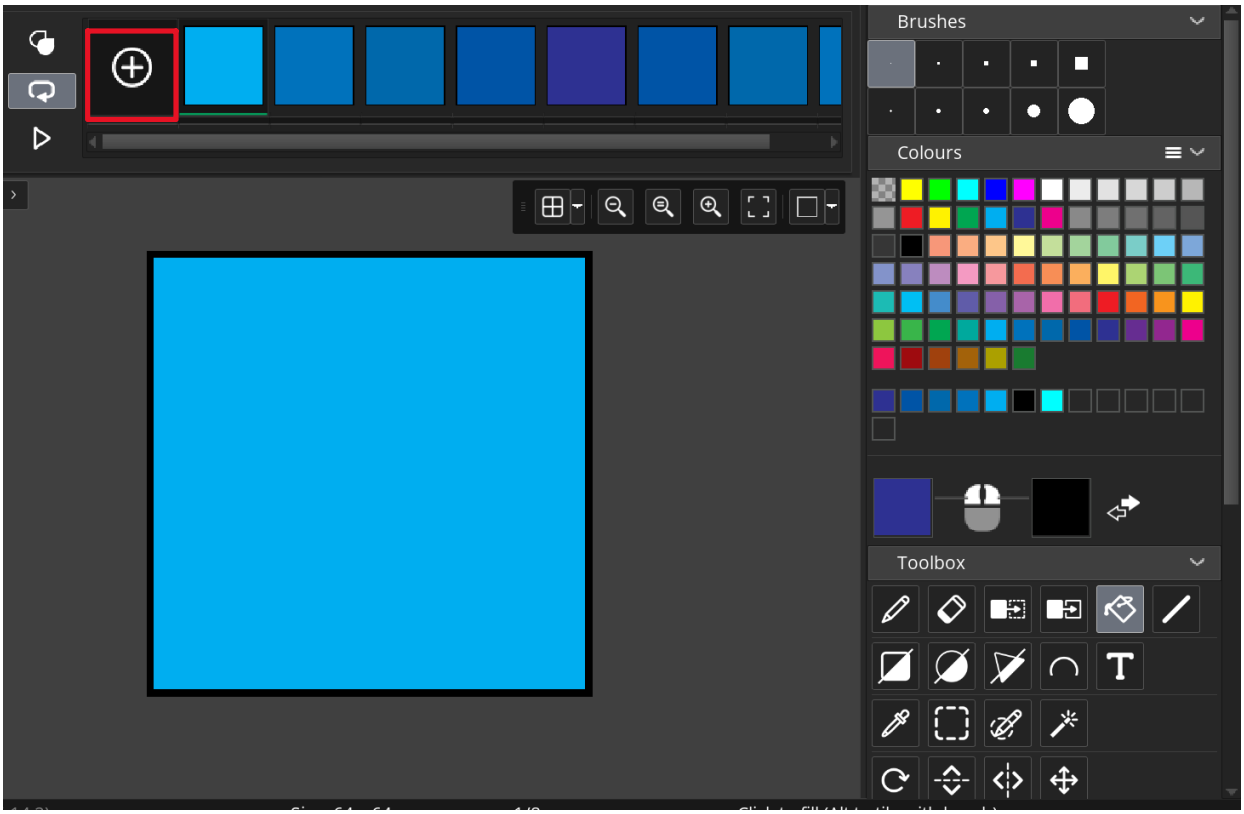

Figure 49 - Adding Frames to Sprite Logo

5. Create a new object in your assets browser. Name it "obj_ScreenSaver". Double-click it to open the object in your workspace.

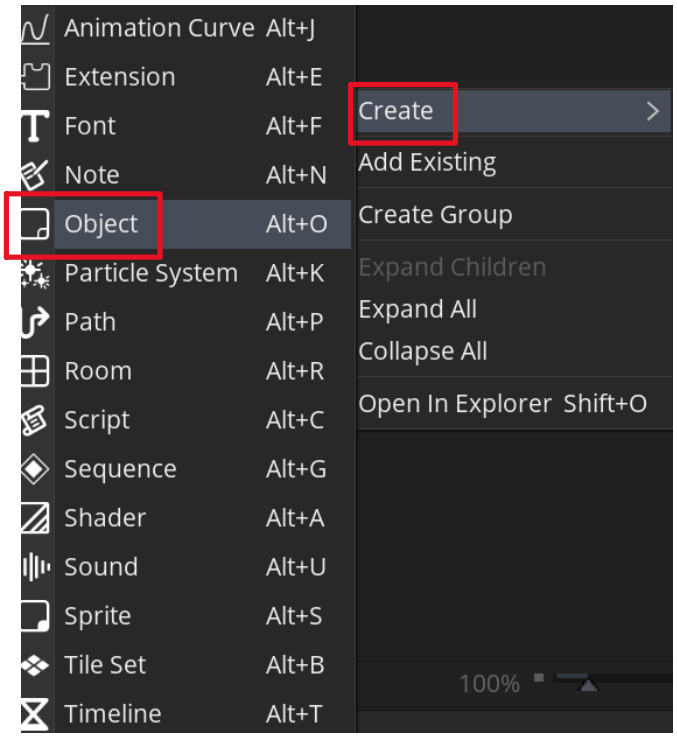

Figure 50 - Creating ScreenSaver Object

6. Drag your spr_Logo sprite into the object's sprite field.

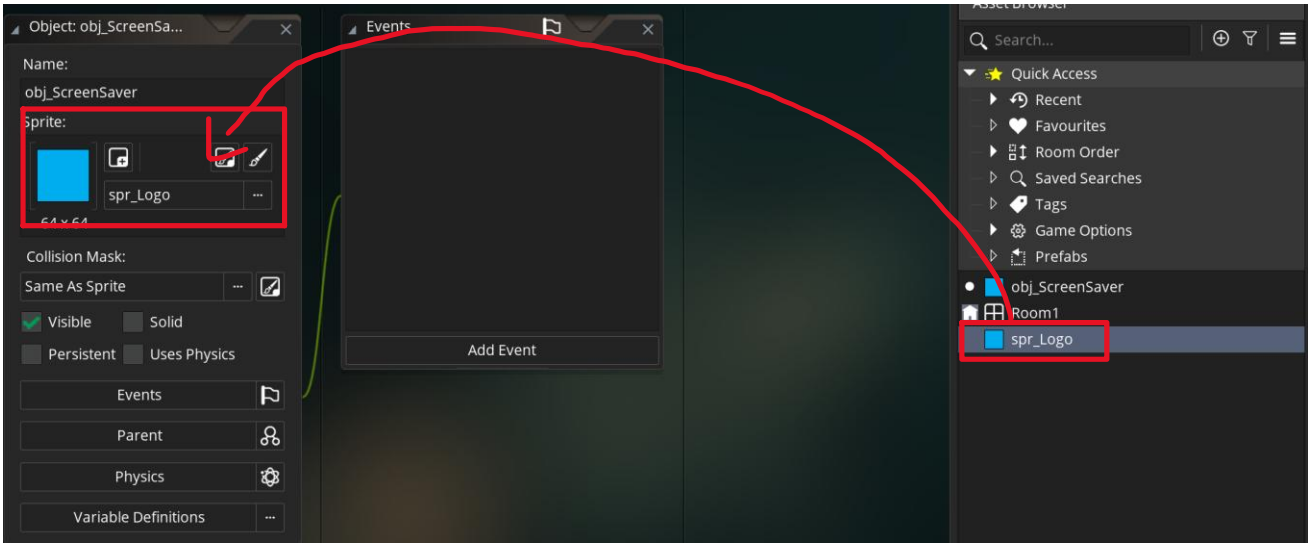

Figure 51 - Assigning Sprite Logo to Object's Sprite

7. We will add two events for our object. Add the Create and Step event. If prompted to choose a coding setup, use the GML Visual setup.

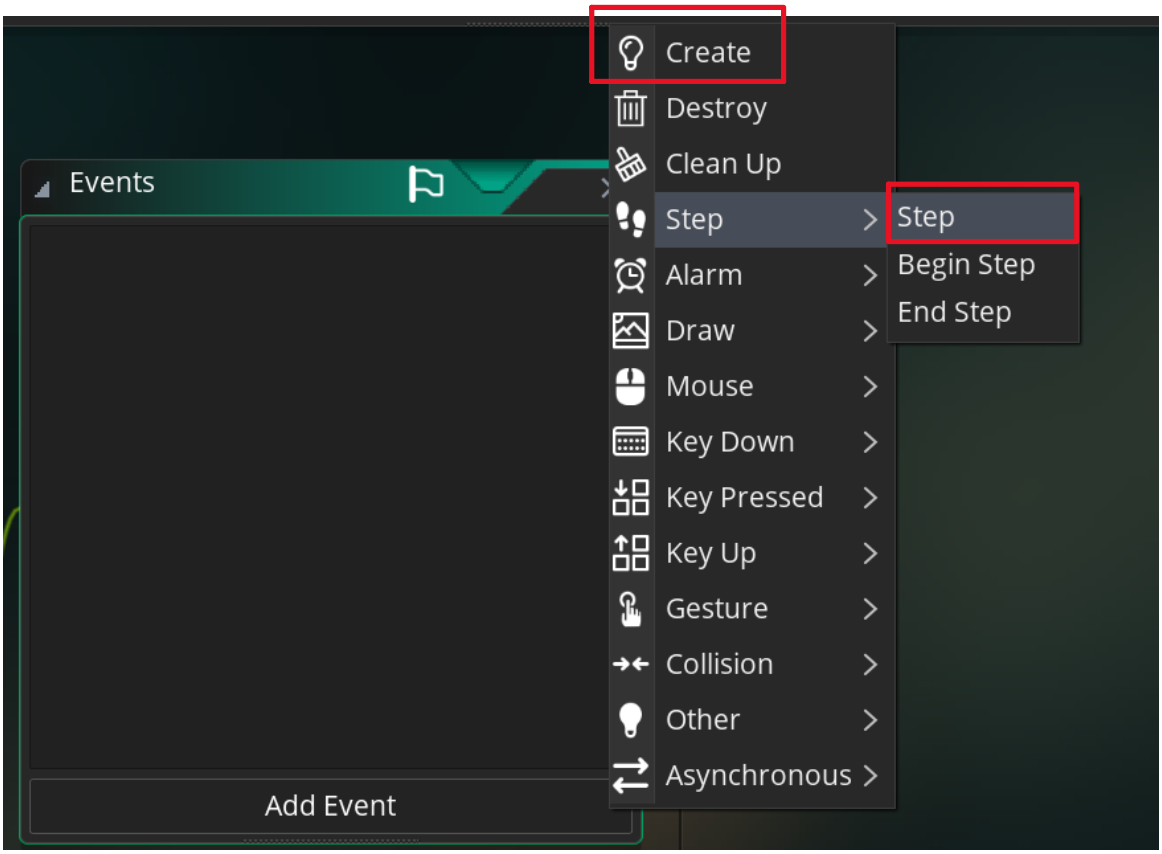

Figure 52 - Adding The Create & Step Event to the ScreenSaver Object

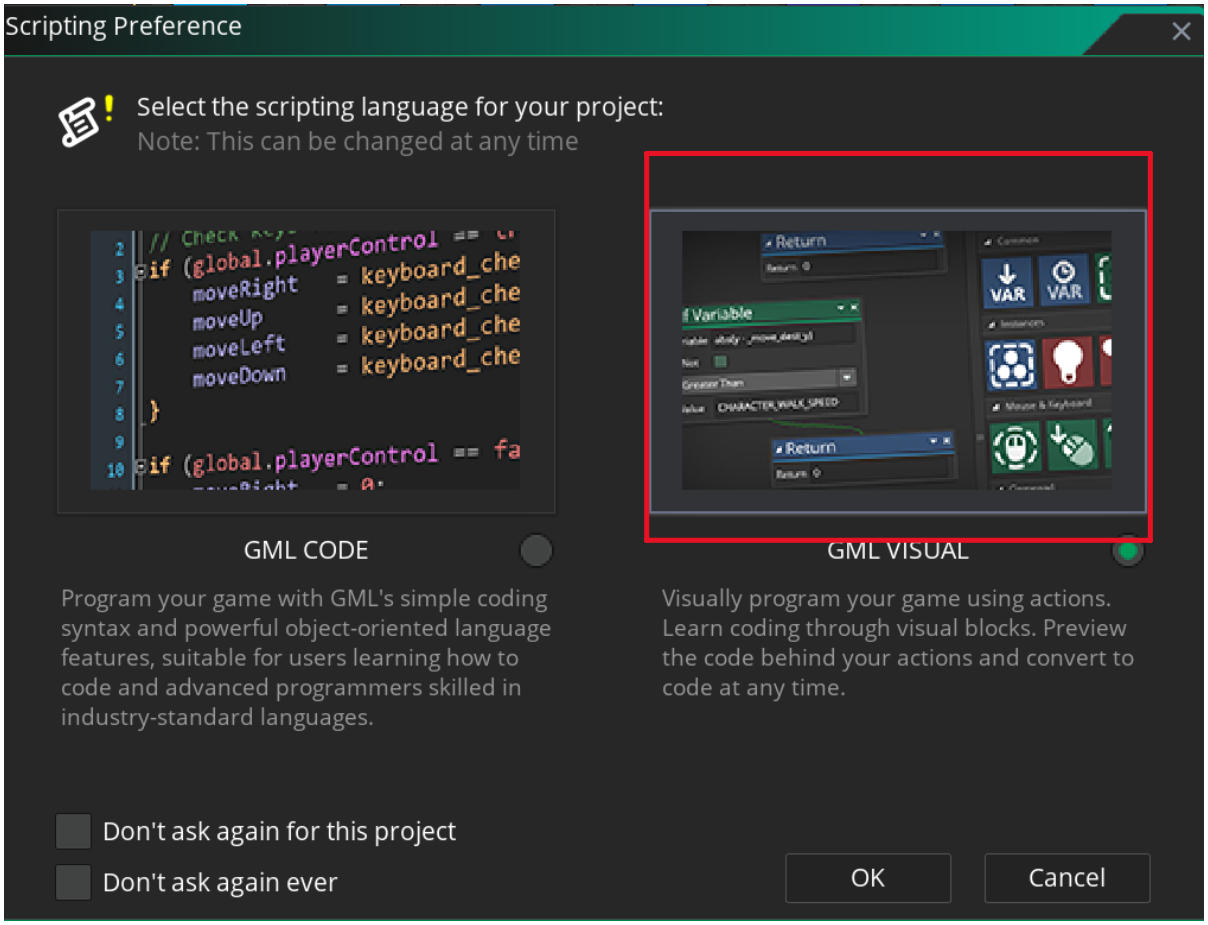

Figure 53 - Selecting GML Visual as Scripting Language for The Object

8. Double-click on the Create event, and add the element below to your event. Be sure to remember your variable's name, as we must reference it exactly in our Step event. Also, be sure the animation speed is set to 0

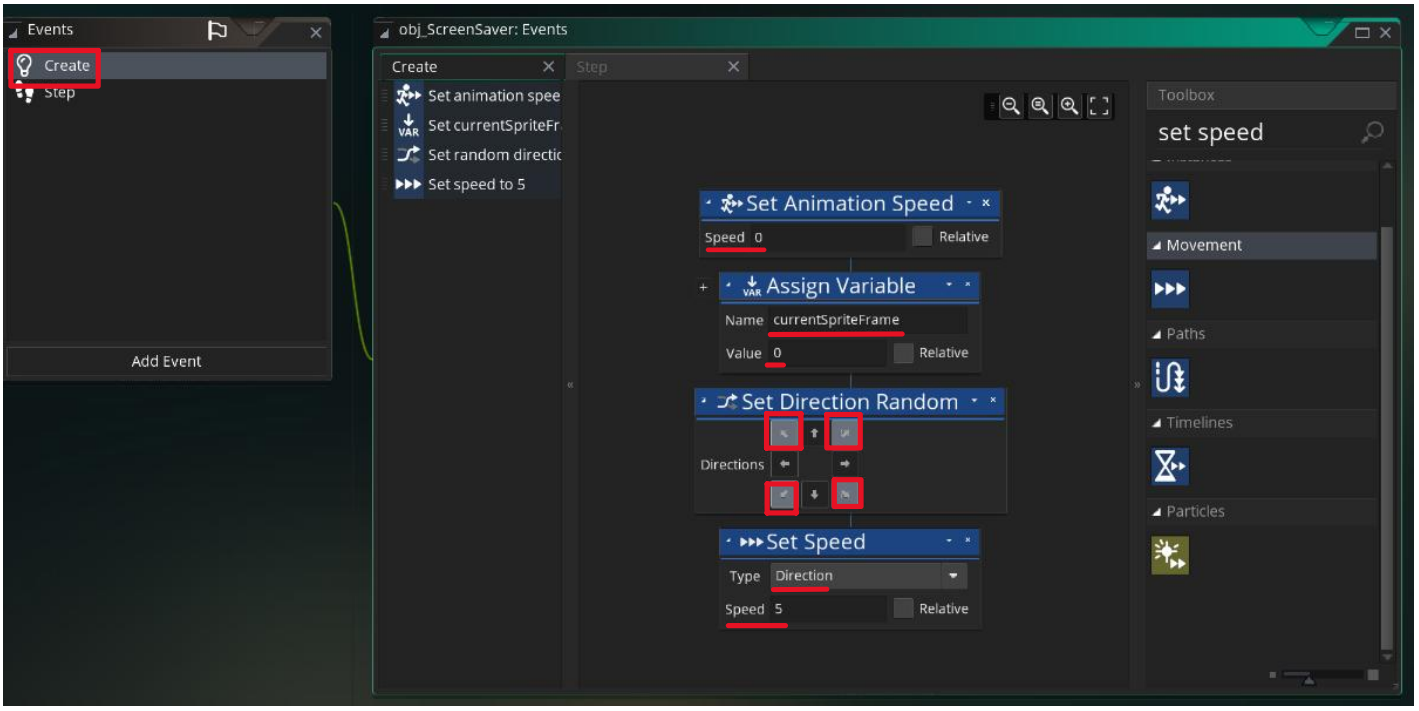

Figure 54 - Setting Up Create Event

9. Double-click on your Step event. This will require more setting up to get it right. First, add one Assign Variable and four IF Variable elements to the Step events. Our Assign Variable element creates a variable that indicates whether we hit the room's border during the step event. Then, each If Variable element will determine if we hit the top, bottom, left, or right border of the wall.

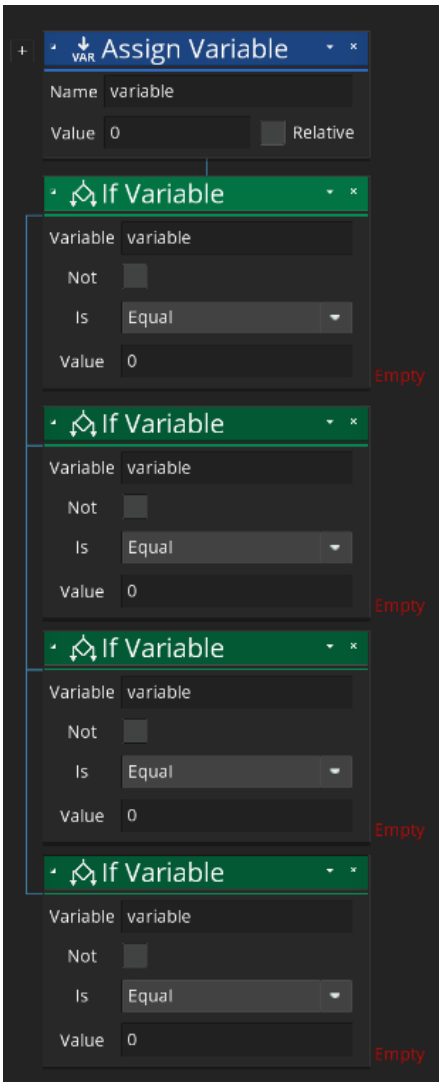

Figure 55 - Initial Setup of Step Event

10. In the Assign Variable element, give it the name “hitBoundaryThisStep”. Set the value to “false”.

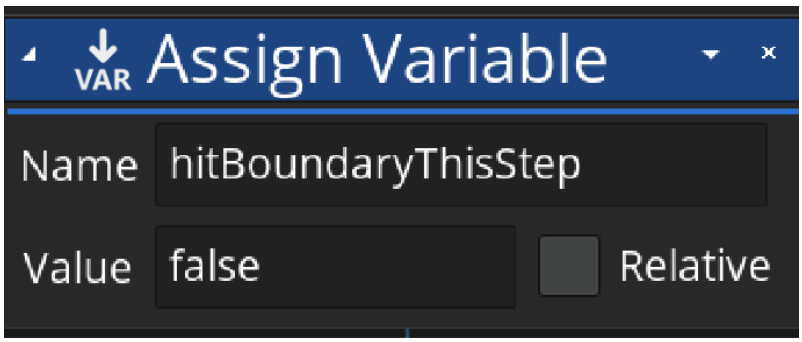

Figure 56 - Configuring The Assign Variable

11. Navigate to your IF Variable elements. Configure each element accordingly:

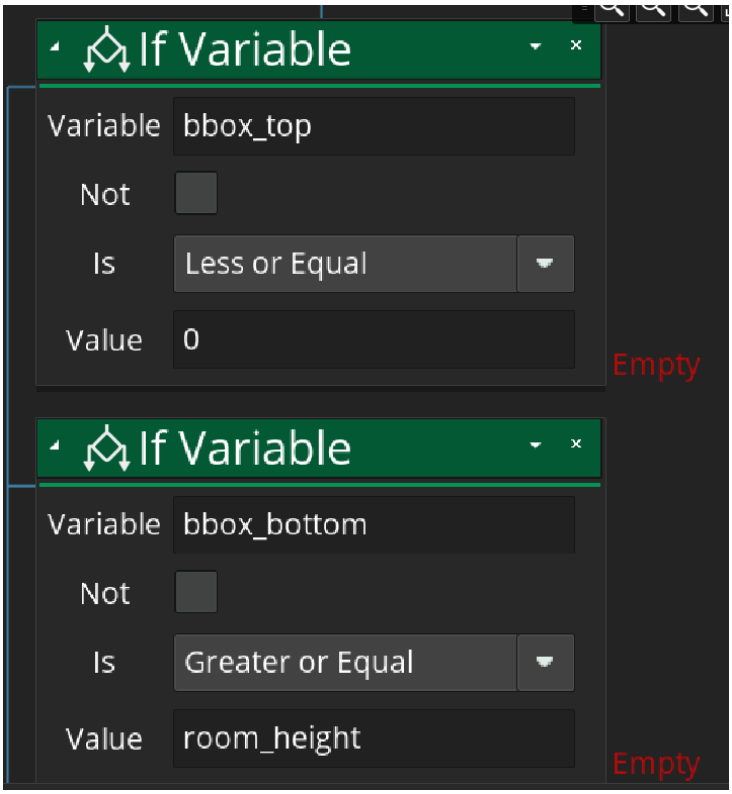

Figure 57 - Configuring IF Variable Element - Part 1

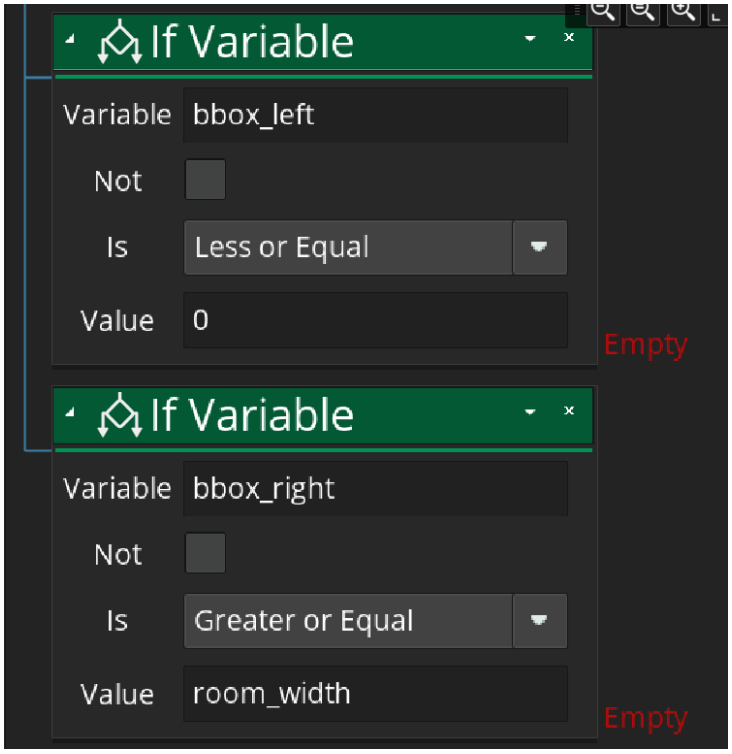

Figure 58 - Configuring IF Variable Elements - Part 2

12. In each IF Variable element, we will run a Set Speed and Assign Variable element. When we hit the top/bottom of the room, we will set the vertical speed of our object to go in the opposite vertical direction. When we hit the left or right wall, we will set the speed of our object to move in the opposite horizontal direction. In each element, we will set our hitBoundaryThisStep variable to true, since we have indeed hit the room's boundary.

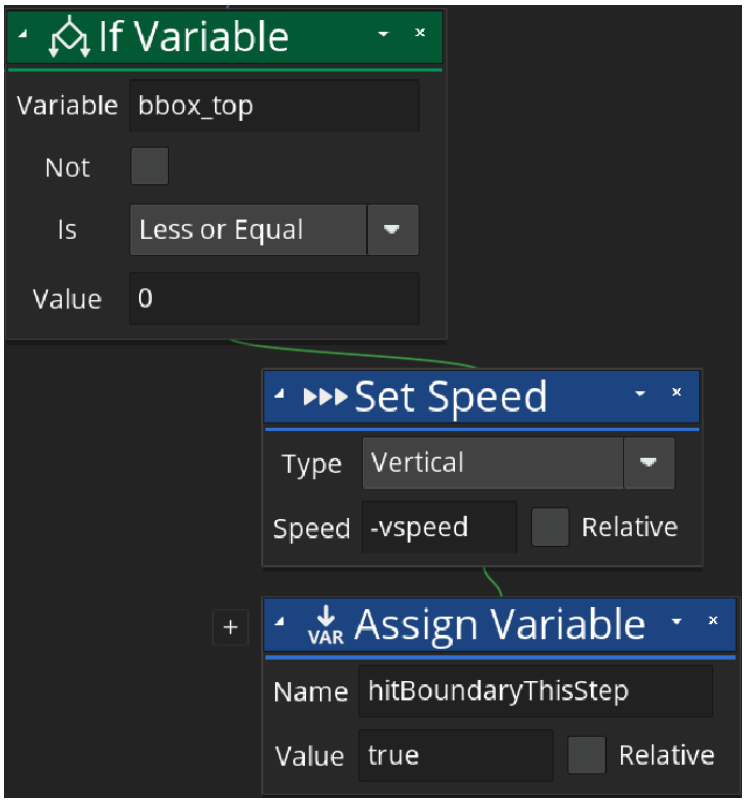

Figure 59 - Setting Up Elements for Hitting the Top of the Room

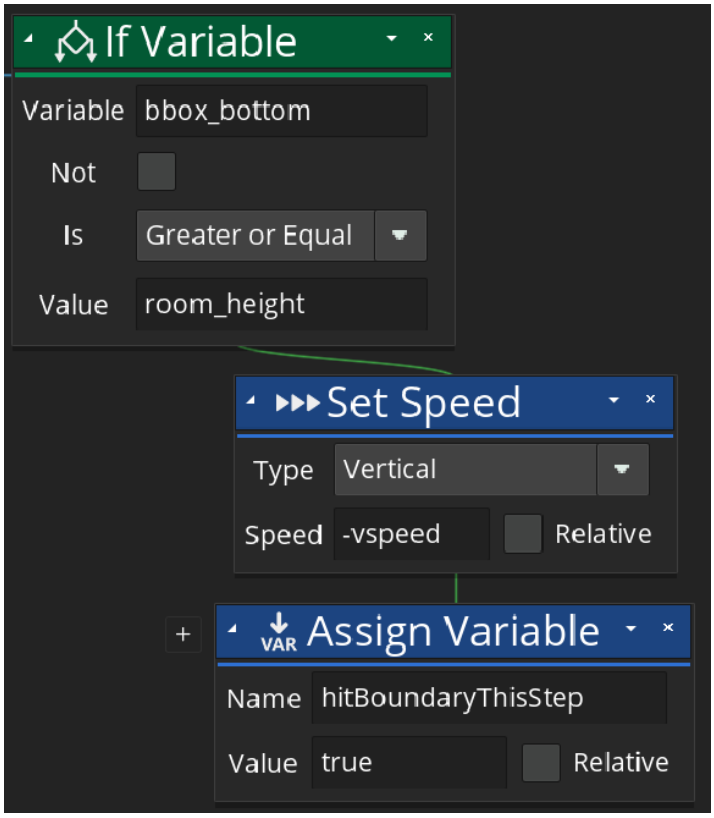

Figure 60 - Setting Up Elements for Hitting Bottom of Room

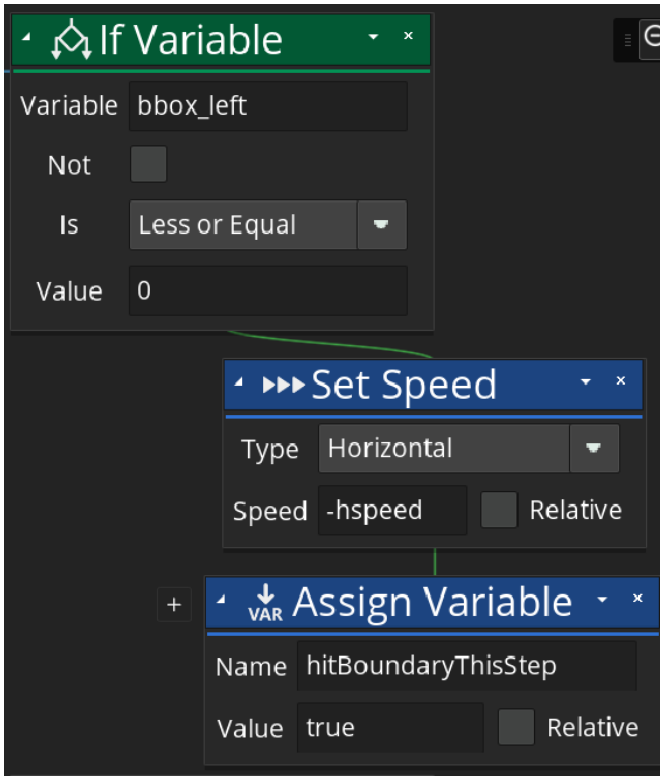

Figure 61 - Setting Up Elements for Hitting Left of Room

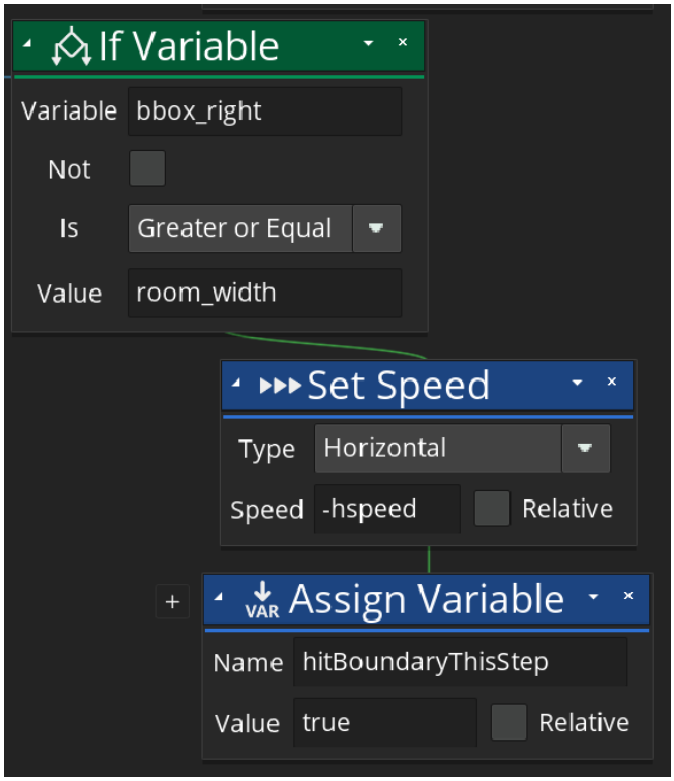

Figure 62 - Setting Up Elements for Hitting the Right of the Room

13. Navigate to the bottom of your Step event. Add another IF Variable to the end. We will check if we hit a wall, and tell our sprite to use its next frame.

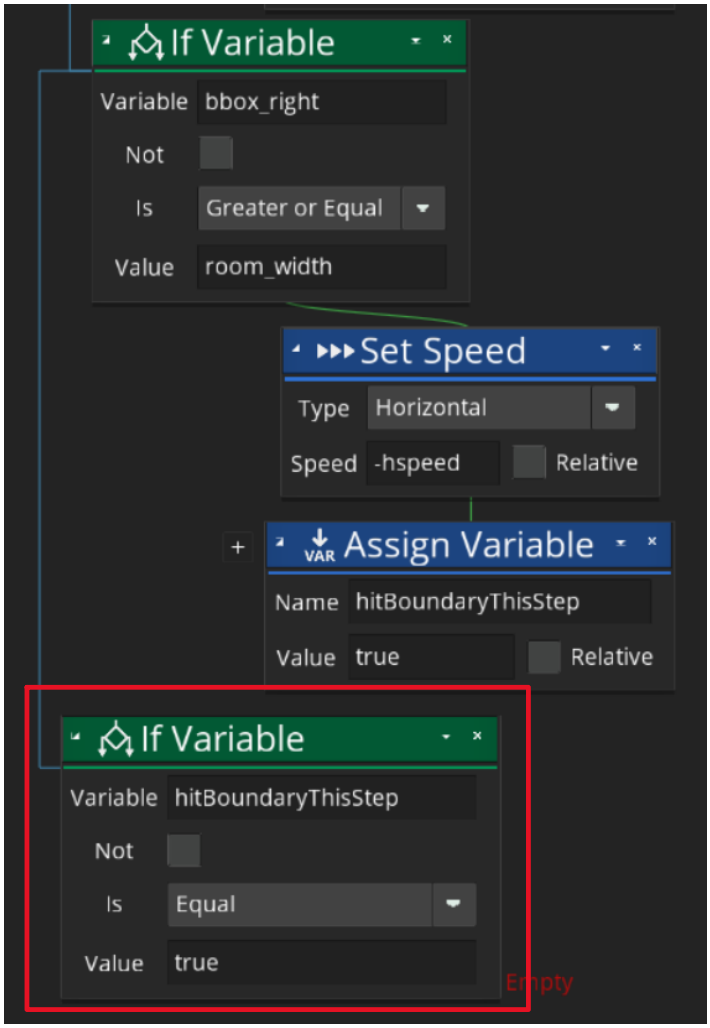

Figure 63 - Adding Check for Hitting Wall During Step Event

14. Add the following element to the IF Variable element. Be sure to apply the values properly:

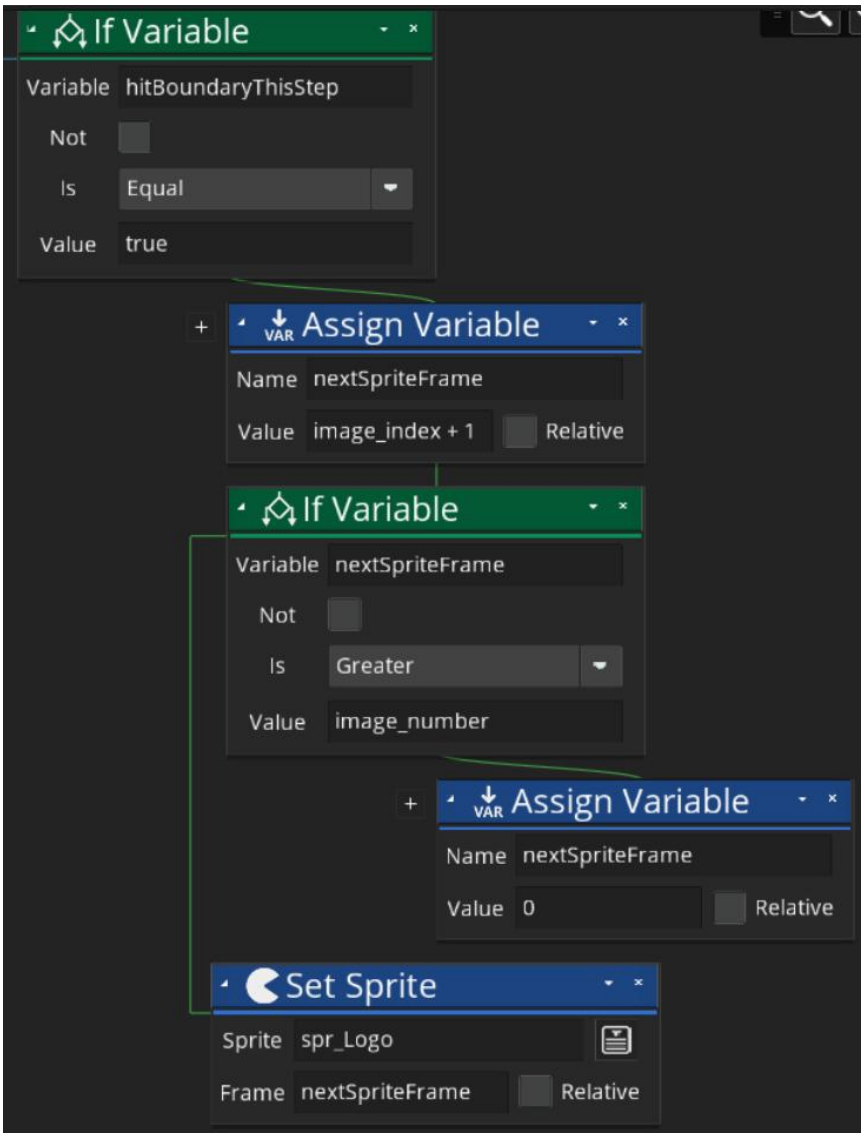

Figure 64 - Changing Sprite Frame When Hitting a Wall

15. Navigate to your assets browser. Double-click on the room asset. Then, drag your obj_ScreenSaver object into the room.

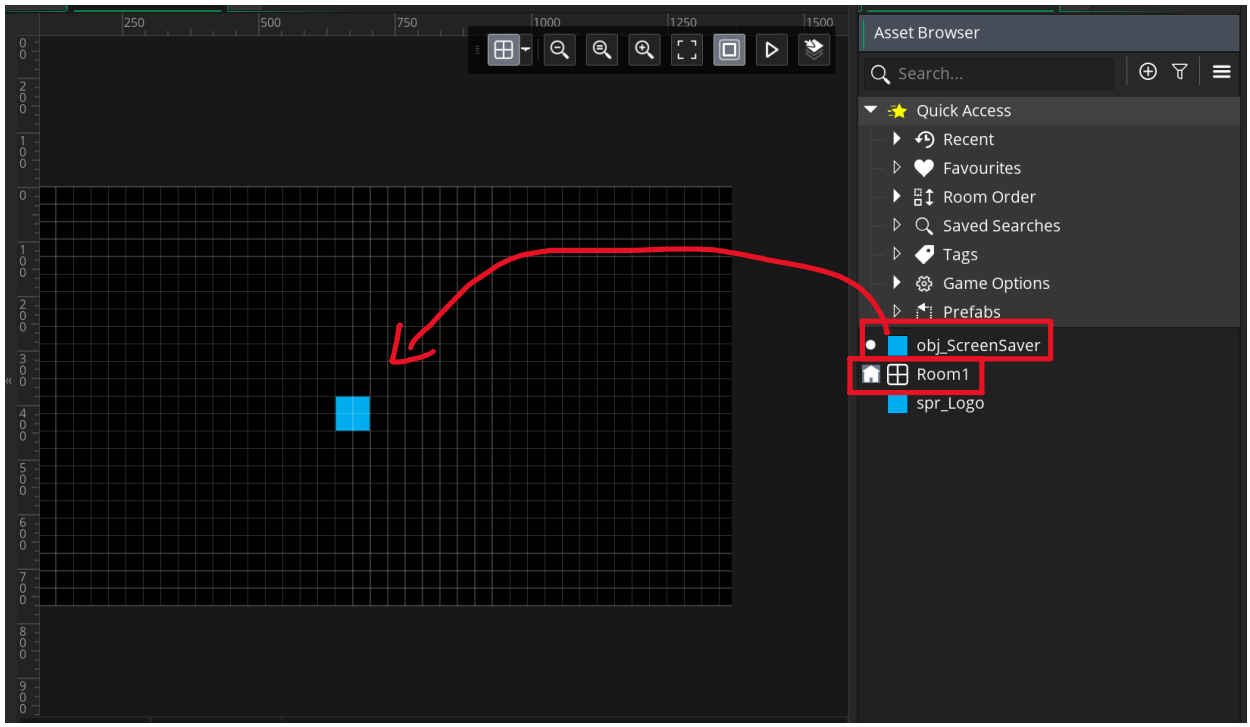

Figure 65 - Adding Our Object to The Room

16. Click play, and observe your screen saver.

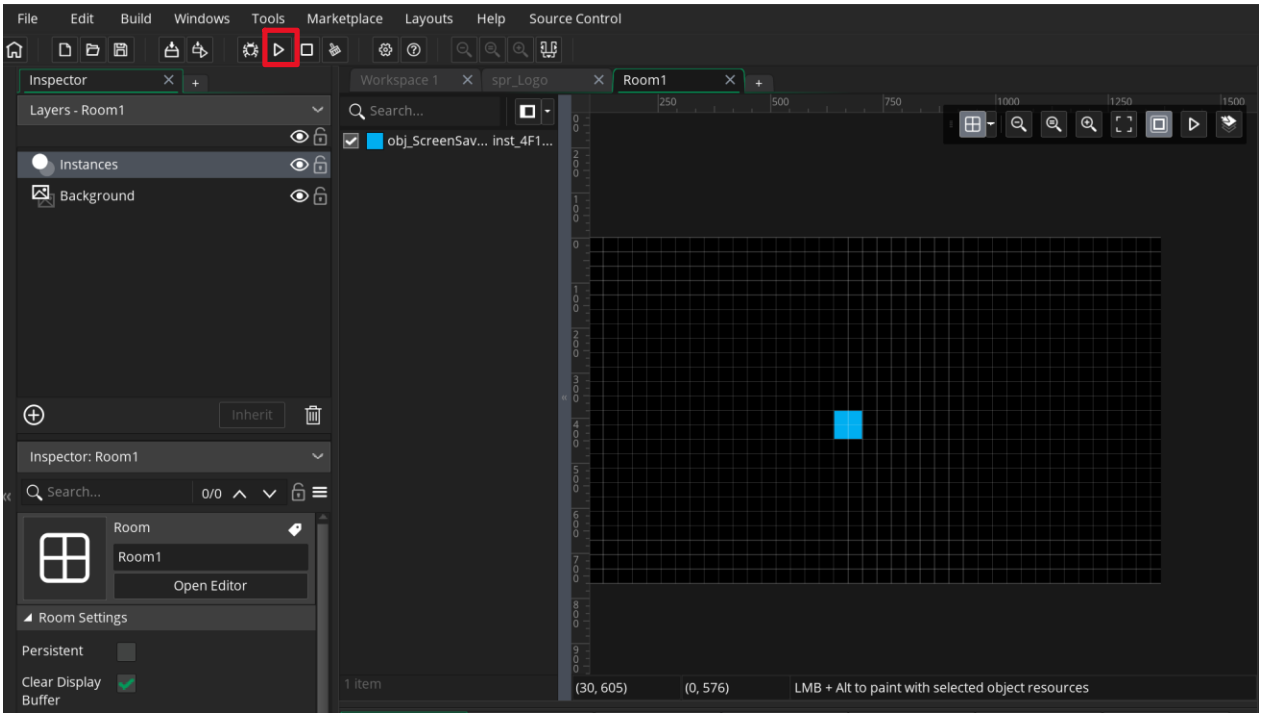

Figure 66 - Location to Run the GameMaker Game

Figure 67 - Sample View of Object Moving in Room

References

https://manual.gamemaker.io/monthly/en/#t=The_Asset_Editors%2FSprites.htm

https://manual.gamemaker.io/monthly/en/#t=The_Asset_Editors%2FObjects.htm

https://manual.gamemaker.io/monthly/en/#t=The_Asset_Editors%2FObject_Properties%2FPhysics_Objects.htm

https://manual.gamemaker.io/monthly/en/#t=The_Asset_Editors%2FRooms.htm